2026-2 중간고사 대비

컴퓨터 구조 · 중간 학습 노트

단원 4개 · 레슨 22개 · 트레이너 학습 탭과 같은 내용

U1 Ch.1 컴퓨터 개요와 성능

L1-1 · 컴퓨터 구성 요소 · 추상화 · ISA

L1-2 · 성능 정의 · 실행 시간 · 클럭

L1-3 · Instruction Count · CPI · 성능 식

L1-4 · 전력 · Amdahl · MIPS 함정 · RISC vs CISC

U2 Ch.2 명령어: 컴퓨터의 언어 (MIPS)

L2-1 · 산술 명령어 · 레지스터 피연산자

L2-2 · 메모리 피연산자 · 엔디안 · lw/sw

L2-3 · 상수 · 2의 보수 · 부호 확장 · 16진수

L2-4 · 명령어 인코딩: R-format · I-format

L2-5 · 논리 연산 · 조건 분기 · if/while

L2-6 · 프로시저 호출 · 스택

L2-7 · 주소 지정 · 32-bit 상수 · 번역과 링크

U3 Ch.3 컴퓨터 연산 (정수 · 부동소수점)

L3-1 · 덧셈 · 뺄셈 · 오버플로

L3-2 · 곱셈 하드웨어

L3-3 · 나눗셈 하드웨어

L3-4 · IEEE 754 부동소수점 표현

L3-5 · 부동소수점 덧셈 · 곱셈 · FP 명령어

U4 Ch.4 프로세서: 데이터패스와 제어 (단순 버전)

L4-1 · 프로세서 개요 · 명령어 실행 단계

L4-2 · 논리 설계 기초 · 클럭

L4-3 · 데이터패스 구성 요소

L4-4 · 전체 데이터패스 따라가기 (R-type · lw · sw · beq)

L4-5 · ALU control

L4-6 · Main control unit · 제어 신호 표

U1 · week1-orientation.pdf · W02 Performance.pdf

Ch.1 컴퓨터 개요와 성능

시험은 영어로 쓰고 영어로 답해. 이 단원은 서술형(용어 정의)과 계산형(CPU time · CPI · Amdahl · MIPS)이 반반. 계산 문제는 교수님이 "수업 예제랑 거의 똑같이 나온다"고 했으니 슬라이드 예제를 손으로 다 풀 수 있어야 해.

컴퓨터 구성 요소 · 추상화 · ISA

교안 범위: W02 p.2–16

§ 수업 범위와 시험 방식

- 교재: Computer Organization and Design (MIPS). 하지만 **수업 자료만으로** 출제.
- **모든 수업·시험이 영어. 답도 영어로 써야 함.**
- 평가: 출석(랜덤 호명) 10%, **중간 45%**, 기말 45%. 중간·기말 모두 closed book, 오프라인.
- 중간 범위: Performance → Instructions → Arithmetic → Processor(단순 버전, single-cycle). 파이프라인은 기말.

시험 포인트: 중간은 10/20(화) 17:00–19:00. 못 오면 교수님께 이메일.

수업 메모 01주-1
모든 수업과 시험이 영어고 **답도 영어로** 써야 한다. 중간·기말 모두 closed book·오프라인. '중간 전까지는 비교적 쉽고, 중간 이후 파이프라인부터 어려워진다.' 출석은 매번 안 부르고 랜덤 호명.

§ 컴퓨터의 종류와 8가지 아이디어

종류	목적	특징
Supercomputer	고성능 과학·공학 계산	성능 지표 **FLOPS** (> MIPS), ~10^17
Server	네트워크 기반 고성능 처리	용량·성능·신뢰성
Personal computer	범용	비용/성능 trade-off
Embedded	시스템 안에 숨어 있음	전력·성능·비용 제약이 가장 엄격

Eight Great Ideas: Design for Moore's Law · Use abstraction to simplify design · **Make the common case fast** · Hierarchy of memories · Dependability via redundancy · Performance via parallelism · Performance via pipelining · Performance via prediction.

수업 메모 01주-2
슈퍼컴퓨터는 FLOPS로 재고 일반 데스크톱보다 약 1만 배 빠르다. 8가지 아이디어 중 memory hierarchy·prediction은 기말 쪽, parallelism·pipelining은 파이프라인과 연결.

§ 프로그램 아래의 계층 · 번역 단계

- **Application software**: 고급 언어(HLL)로 작성 (브라우저, 앱, Office).
- **System software**: **Compiler** (HLL → 기계어, 예: GCC) + **Operating System** (입출력, 메모리·저장장치 관리, 작업 스케줄링·자원 공유).
- **Hardware**: processor, memory, I/O controllers.

번역: `A+B` (HLL) → `add A, B` (assembly, 사람도 읽음) → 이진 machine language.

OS가 앱에 해 주는 것: loading·process creation, **system call**, resource management(메모리·CPU 스케줄링·I/O), IPC, event handling, error handling. 슬라이드 내용을 외울 필요는 없고 "OS가 앱 흐름에서 무엇을 하는지"만.

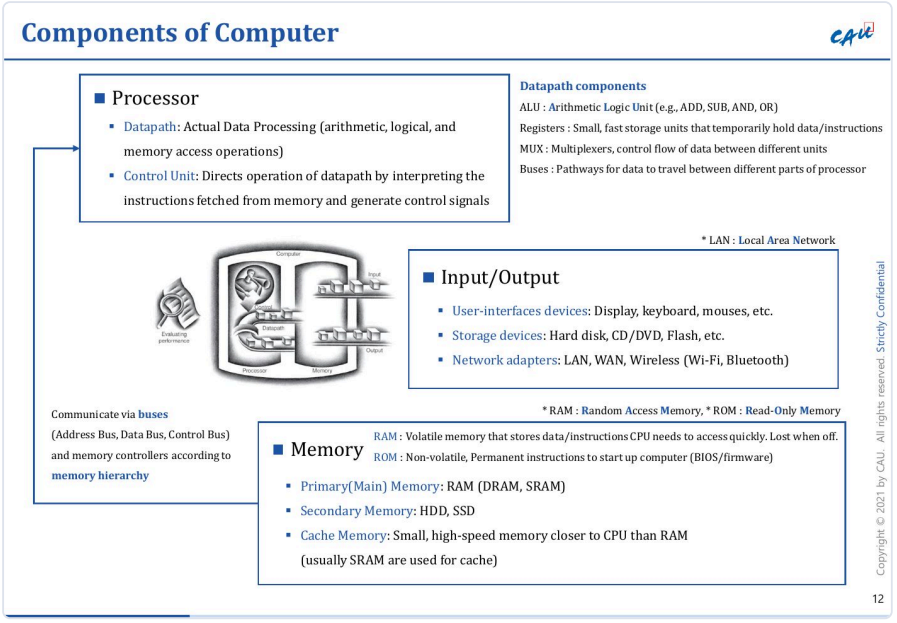
> **수업 메모** 01주-2
> OS 슬라이드는 외울 필요 없고 'OS가 앱 흐름에서 무엇을 하는지'만. 예: 카톡에서 사진 보낼 때 '접근 허용하시겠습니까?'가 뜨는 게 OS의 일(system call).

§ 컴퓨터의 구성 요소

- **Input/Output**: 사용자 장치(키보드·마우스), 저장 장치(HDD·Flash), **네트워크 어댑터도 I/O** (LAN·Wi-Fi·Bluetooth).
- **Memory**: Primary(RAM: DRAM/SRAM, 휘발성), Secondary(HDD·SSD), **Cache**(작고 빠름, 보통 SRAM). ROM = 비휘발, 부팅용 펌웨어(BIOS).
- **Processor = Datapath + Control unit**
  - **Datapath**: 실제 데이터 처리(산술·논리·메모리 접근). 구성: ALU, Registers, MUX, Buses.
  - **Control unit**: 메모리에서 가져온 명령어를 해석해 **control signals**를 만들어 datapath를 지휘.
- 장치끼리는 **bus**(address bus, data bus, control bus)로 통신.

> 시험 포인트: "Processor의 두 구성 요소와 역할" → Datapath(데이터를 실제로 처리), Control unit(명령어 해석 → 제어 신호 생성).

> **수업 메모** 01주-2
> 네트워크 장치도 I/O라는 걸 많이 헷갈린다. Control unit이 명령어(예: 덧셈)를 읽으면 datapath의 ALU가 동작하도록 신호를 보낸다. 캐시는 이 수업에서 가장 주의할 메모리(기말).



W02 p.13 · Components of Computer: I/O, Memory, Processor(Datapath + Control)

§ 추상화와 ISA

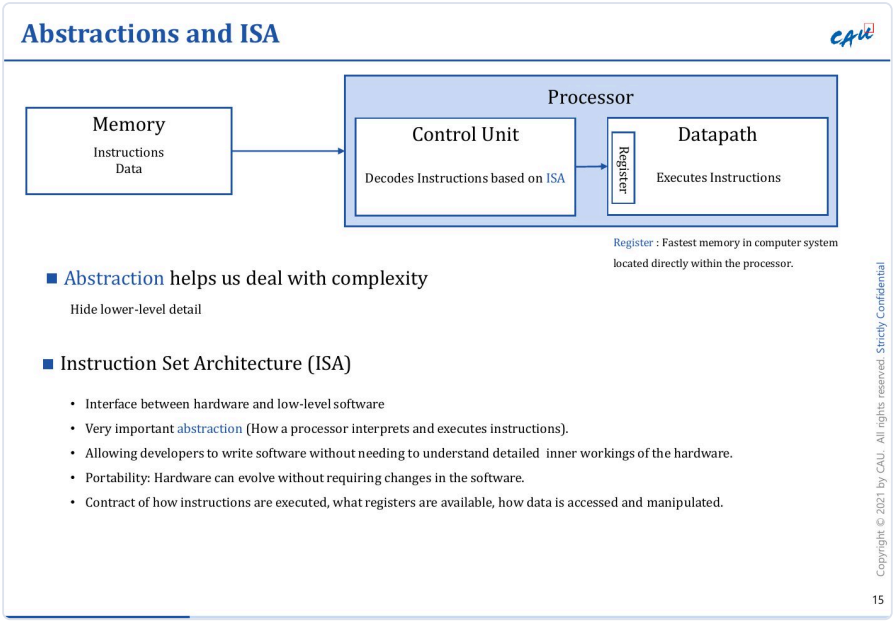
ISA (Instruction Set Architecture): 하드웨어와 저수준 소프트웨어 사이의 **인터페이스**. 프로세서가 명령어를 어떻게 해석·실행하는지, 어떤 레지스터가 있는지, 데이터에 어떻게 접근하는지에 대한 **계약(contract)**.

- 개발자가 하드웨어 내부를 몰라도 소프트웨어를 쓸 수 있게 해 줌 (abstraction = hide lower-level detail).
- **Portability**: 하드웨어가 바뀌어도 소프트웨어를 고칠 필요가 없음. 교수님이 가장 중요하다고 한 성질.

- Register = 프로세서 안에 있는 가장 빠른 메모리.

한 줄 답안: ISA is the interface between hardware and low-level software; it defines how instructions are executed, which registers exist, and how data is accessed, so software can run without knowing hardware details (portability).

수업 메모 01주-2
커피 심부름 비유: '커피 사 와' 한마디 안에 '문 열고 → 엘리베이터 → 1층 → 주문 → 결제 → 다시 올라오기'가 다 숨어 있다. ISA가 그런 하드웨어 세부를 숨겨 준다. 가장 중요한 성질은 **portability**.



W02 p.16 · Abstractions and ISA

성능 정의 · 실행 시간 · 클럭

교안 범위: W02 p.17–26

§ 성능을 좌우하는 요소

요소	영향
Algorithm	실행할 **연산 수** (예: bubble sort n^2 vs merge sort n log n)
Programming language, compiler, architecture	연산당 **기계 명령어 수**
Processor & memory system	명령어가 얼마나 빨리 실행되나
I/O system (OS 포함)	I/O 연산이 얼마나 빨리 실행되나

Moore's Law: IC capacity가 18–24개월마다 두 배. 하지만 전력 제약(power wall) 때문에 성능이 무한히 오르진 않음.

> **수업 메모** 01주-2 · 02주-1
> n log n 알고리즘을 n^2로 바꾸면 프로세서가 아무리 빨라도 느려진다. 총 기계 명령어 수 = 연산 수 × 연산당 기계 명령어 수. 같은 코드도 보통 C가 Python보다 빠르다.

§ Response time vs Throughput

- **Response time** (= latency = execution time): 한 작업을 끝내는 데 걸리는 시간.
- **Throughput**: 단위 시간당 처리하는 작업량 (tasks/hour).
- 수업에서는 **response time 기준으로 통일**.

> **수업 메모** 01주-2
> response time은 스톱워치로 한 작업이 끝날 때까지 재는 것, throughput은 단위 시간 동안 몇 개를 처리하나. 수업은 response time으로 통일.

§ 성능 비교: X is n times faster than Y

`Performance_X = 1 / Execution time_X`

`Performance_X / Performance_Y = Execution time_Y / Execution time_X = n` → "X is n times faster than Y".

예제(슬라이드): 같은 프로그램이 A에서 7s, B에서 21s → Perf_A/Perf_B = 21/7 = **3**, A가 B보다 3배 빠름.

함정: 비율을 뒤집지 마. 성능 비 = 시간 비의 **역수**. 교수님이 "많은 학생이 시험에서 이걸 틀린다"고 두 번 강조함.

> **수업 메모** 01주-2 · 02주-1
> 'many students miss this in their midterm exams' — 성능 비는 실행 시간 비의 **역수**. 7 s vs 21 s 예제를 2분 동안 직접 풀게 하고 답 3을 확인.

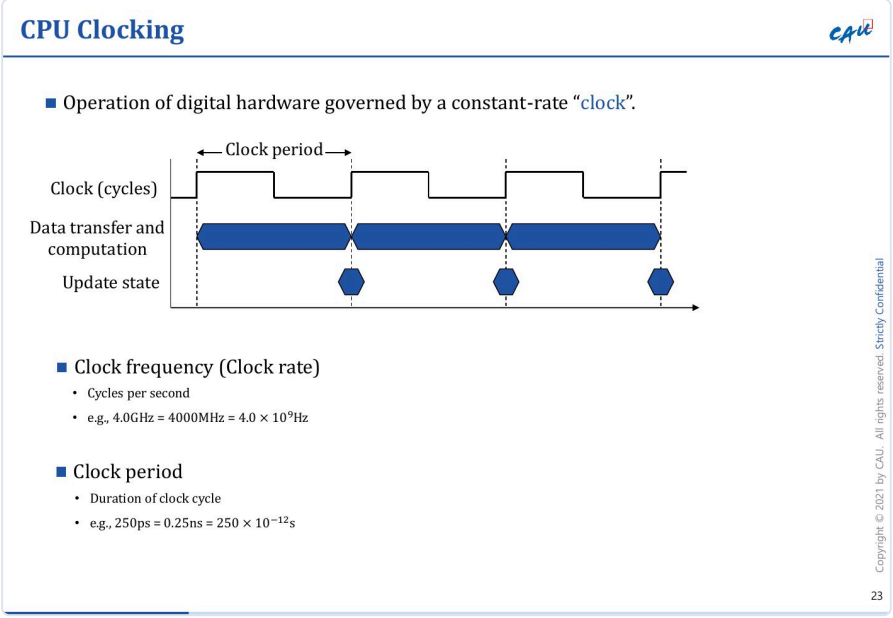
§ Elapsed time · CPU time · Clock

- **Elapsed time**: 전체 응답 시간 (I/O 대기, OS 오버헤드 포함). 시스템 성능을 보여 주지만 비교용으로는 부정확.
- **CPU time** = user CPU time + system CPU time.

- **User CPU time**: 우리 프로그램의 명령어를 실제로 실행한 시간 (user mode). **수업에서 execution time = user CPU time.**
- System CPU time: kernel mode (하드웨어 접근, I/O, 메모리 관리).

- **Clock**: 0→1→0으로 일정 주기로 바뀌는 내부 신호.
  - **Clock period** (cycle time): 한 사이클 길이, 예) 250 ps.
  - **Clock rate** (frequency): 초당 사이클 수, 예) 4.0 GHz = 4.0×10^9 Hz. **둘은 역수 관계** (4 GHz ↔ 250 ps).

> **수업 메모** `02주-1`
> elapsed time은 I/O 지연, 메모리→프로세서 이동 지연까지 들어가서 비교에 부정확. 그래서 CPU 시간, 그중 **user CPU time** 만 본다. 앞으로 '실행 시간'이라고 하면 user CPU time.



W02 p.24 · CPU Clocking: clock period와 clock rate

§ CPU Time 공식과 예제

`CPU Time = CPU Clock Cycles × Clock Cycle Time = CPU Clock Cycles / Clock Rate`

성능을 올리려면: clock cycle 수를 줄이거나, clock rate를 올림. 둘은 trade-off.

슬라이드 예제: A는 2 GHz에서 10 s. B는 6 s가 목표, 대신 clock cycle이 A의 1.2배.

```
Cycles_A = 10 s × 2×10⁹ Hz = 20×10⁹
Cycles_B = 1.2 × 20×10⁹ = 24×10⁹
Clock rate_B = 24×10⁹ / 6 s = 4×10⁹ Hz = 4 GHz
```

> 시험 포인트: 교수님 말 그대로 "the examples that I provided during class will resemble very much your midterm". 식을 변형해서 모르는 값을 구하는 연습.

> **수업 메모** `02주-1`
> 'the examples that I provided during class will resemble very much your midterm or your final exam' — 예제를 풀 줄 알면 시험 문제도 풀 수 있다고 함. clock rate = cycles / CPU time으로 식을 돌려서 풀이.

Instruction Count · CPI · 성능 식

교안 범위: W02 p.27–31

§ IC · CPI · 세 요소 공식

- **Instruction Count (IC)**: 프로그램이 실행하는 명령어 수. program, ISA, compiler가 결정.
- **CPI (Cycles Per Instruction)**: 명령어당 평균 클럭 사이클. CPU 하드웨어가 결정. 명령어마다 다르면 평균 CPI.

`Clock Cycles = IC × CPI`

`CPU Time = IC × CPI × Clock Cycle Time = (IC × CPI) / Clock Rate`

슬라이드 예제: A는 cycle 250 ps·CPI 2.0, B는 500 ps·CPI 1.2, 같은 프로그램(IC = I).

```
CPU time_A = I × 2.0 × 250 ps = 500·I ps
CPU time_B = I × 1.2 × 500 ps = 600·I ps
Perf_A / Perf_B = 600I / 500I = 1.2  → A is 1.2 times faster than B
```

시험 포인트: 모르는 IC는 변수 I로 두면 비율에서 약분된다. 교수님이 풀이에서 그대로 쓴 방법.

수업 메모 02주-1
IC를 모르면 같은 변수 i로 두면 두 컴퓨터를 나눌 때 약분된다. CPI는 CPU 성능의 가장 직접적인 척도.

§ 가중 평균 CPI

명령어 클래스마다 CPI가 다르면:

`Clock Cycles = Σ (CPI_i × IC_i)`

`Average CPI = Clock Cycles / IC = Σ CPI_i × (IC_i / IC)` (IC_i / IC = relative frequency)

슬라이드 예제 (CPI: A=1, B=2, C=3):

	A	B	C	총 IC	Cycles	평균 CPI
Sequence 1	2	1	2	5	2+2+6 = 10	**2.0**
Sequence 2	4	1	1	6	4+2+3 = 9	**1.5**

Sequence 2가 명령어는 더 많지만 사이클은 더 적어서 빠르다.

수업 메모 02주-1
풀이 순서: 총 명령어 수부터 세고 → 클래스별 CPI × IC를 더하고 → 총 명령어 수로 나눈다.

§ 성능 요약표

`CPU Time = (Instructions / Program) × (Clock cycles / Instruction) × (Seconds / Clock cycle)`

	IC	CPI	Clock rate
Algorithm/Program	✔	✔	
Programming language	✔	✔	

	IC	CPI	Clock rate
Compiler	✓	✓	
ISA	✓	✓	
Hardware organization		✓	✓
Technology			✓

함정: Hardware organization은 CPI와 clock rate에 영향, IC에는 영향 없음. Technology는 clock rate만.

Performance Summary

■ The final CPU Performance (=CPU Time) is written as:

$$\text{CPU Time} = \left(\frac{\text{Instructions}}{\text{Program}}\right) \times \left(\frac{\text{Clock cycles}}{\text{Instruction}}\right) \times \left(\frac{\text{Seconds}}{\text{Clock cycle}}\right)$$

■ Performance Depends on

	instruction count	CPI	clock rate
Algorithm/Program	✓	✓	
Programming language	✓	✓	
Compiler	✓	✓	
ISA	✓	✓	
Hardware organization		✓	✓
Technology			✓

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

30

W02 p.31 · Performance Summary 표

전력 · Amdahl · MIPS 함정 · RISC vs CISC

교안 범위: W02 p.32–41

§ Power wall

CMOS: `Power ∝ 1/2 × Capacitive load × Voltage² × Frequency`

예: 새 CPU가 capacitive load 85%, voltage 15%↓, frequency 15%↓ → P_new/P_old = 0.85 × 0.85² × 0.85 = $0.85^4 \approx$ **0.52**.

Power wall: 전압을 더 낮출 수 없고, 열을 더 뺄 수 없음 → 단일 프로세서 성능 향상이 막힘 → **multicore** (explicit parallel programming, load balancing 필요). 교수님: 식은 외울 필요 없고 "전력 제약이 있다"는 것만.

> **수업 메모** 02주-1 · 02주-2
> 식은 외울 필요 없고 '전력은 무한하지 않다'는 것만 가져가라. power wall: 전압을 더 못 낮추고, 열을 더 못 뺀다.

§ Amdahl's Law

Pitfall: 컴퓨터의 한 부분을 개선하면 전체 성능도 그만큼 좋아질 거라고 기대하는 것.

`T_improved = T_affected / improvement factor + T_unaffected`

슬라이드 예제: 100 s 중 곱셈이 80 s. 전체를 5배 빠르게(20 s) 하려면?

```
20 = 80/n + 20  →  80/n = 0  → 불가능
```

곱셈을 무한히 빠르게 해도 나머지 20 s가 병목이라 최대 5배에 그친다. → **Make the common case fast**와 연결.

> **수업 메모** 02주-2
> 곱셈을 무한히 빠르게 해도 나머지 20초가 병목이라 최대 5배. 'this is why our computers are still very slow'.

§ MIPS는 성능 지표로 부적절

`MIPS = IC / (Execution time × 10⁶) = Clock rate / (CPI × 10⁶)`

MIPS는 (1) 컴퓨터마다 ISA가 다른 것, (2) 명령어 복잡도 차이를 반영 못 함. IC가 약분돼서 사라짐.

슬라이드 예제 (둘 다 4 GHz):

	IC	CPI	MIPS	CPU time
A	10 billion	1.0	4000	$10\times10^9\times1.0 / 4\times10^9$ = 2.5 s
B	8 billion	1.1	≈3636	$8\times10^9\times1.1 / 4\times10^9$ = 2.2 s

MIPS로는 A가 높지만, **실제로는 B가 빠름** (Perf_A/Perf_B = 2.2/2.5 = 0.88). 그래서 CPU time을 쓴다.

> **수업 메모** 02주-2
> MIPS는 instruction count가 약분돼서 사라지니 비교가 틀어진다. 예제에서 MIPS는 A가 높지만 CPU time으로 보면 8.8/10 < 1 → B가 빠름.

§ 기타: SPEC · 유휴 전력

- **SPEC** (Standard Performance Evaluation Corp.): CPU·I/O·Web 등의 표준 벤치마크 세트. SPEC CPU는 I/O가 거의 없는 프로그램의 elapsed time, SPEC Power는 부하 수준별 전력.
- **Fallacy: low power at idle** — i7은 10% 부하에서도 121 W(최대의 47%). 데이터센터는 대부분 10–50% 부하 → 전력이 부하에 비례하도록 설계해야.

§ RISC vs CISC

	RISC	CISC
예	**MIPS**, ARM, PowerPC	**x86, x86-64** (Intel, AMD)
전략	단순하고 작은 명령어 집합	크고 복잡한 명령어로 프로그램당 명령어 수 최소화
장점	빠른 실행, 전력 효율, 확장성(병렬화 쉬움)	프로그램 크기 감소, 옛 명령어와 호환
단점	복잡한 작업에 명령어가 더 많이 필요 → 프로그램 커짐	느린 실행, 전력 소모 큼

MIPS = Microprocessor without Interlocked Pipeline Stages. (성능 지표 MIPS와 이름만 같음!)

> **수업 메모** 02주-2 · 03주-1
> 윈도우 시스템 정보의 x86/x64가 CISC라는 표시. RISC는 '문 열고 나가서, 엘리베이터 타고...'처럼 일일이 명령해야 하는 쪽, CISC는 그걸 큰 명령 하나로 묶는 쪽.

U2 · W02 p.41 · week3-수업.pdf · week4-수업.pdf p.2–5

Ch.2 명령어: 컴퓨터의 언어 (MIPS)

시험의 큰 덩어리. "명령어와 설명을 주고 빈칸이나 MIPS 표현을 채우는" 형식이 예고됐어. 교수님이 0점 처리한다고 경고한 규칙이 많은 단원이니 함정 박스를 꼭 읽어.

산술 명령어 · 레지스터 피연산자

교안 범위: week3 p.3–8

§ MIPS 명령어 분류

MIPS: Stanford에서 나온 RISC ISA, 임베디드 시장에서 큰 점유율, 현대 ISA의 전형.

- **종류별**: Arithmetic · Memory(Data Transfer) · Logical · Conditional · Branch/Jump · Floating Point · Pseudo
- **형식별**: **R-format · I-format · J-format** · FP format

§ 산술 명령어 형식

`add a, b, c # a = b + c`

- **destination 1개 + source 2개**. 모든 산술 명령어가 이 형식.
- **Design Principle 1: Simplicity favors regularity** — 규칙적이면 구현이 단순, 단순하면 저비용·고성능.

슬라이드 예제:

```
a = b + c + d;          f = (g + h) - (i + j);
add a, b, c             add t0, g, h
add a, a, d             add t1, i, j
                        sub f, t0, t1
```

함정: 결과는 **사용자가 원한 변수**(여기선 f)에 넣어야 한다. 교수님이 "must be very careful at this term"이라고 강조.

> **수업 메모** 02주-2
> a = b + c + d를 add 하나로 쓰면 안 되고 두 줄로 나눈다. 결과는 반드시 사용자가 원한 변수에 — 'you must take a careful look at where to put your result'.

§ 레지스터 피연산자

- 산술 명령어는 **레지스터 피연산자만** 사용. 레지스터는 프로세서 안의 특별한 저장 위치.
- MIPS: **32개 × 32-bit 레지스터** (번호 0–31). 32-bit = word size (설계 당시 표준, MIPS64에서도 호환성 때문에 유지).
- 어셈블러 이름: `$t0-$t9` (임시), `$s0-$s7` (saved).
- **Design Principle 2: Smaller is faster** — 레지스터를 작게 만들어 자주 쓰는 데이터를 빨리 처리.

슬라이드 예제 (f, g, h, i, j가 $s0–$s4):

```
add $t0, $s1, $s2     # g + h
add $t1, $s3, $s4     # i + j
sub $s0, $t0, $t1     # f = (g+h) - (i+j)
```

함정: 연산 결과는 메모리로 바로 갈 수 없다. 항상 **레지스터에** 저장. (교안 이 슬라이드에 add/sub 오타가 있었고 수업에서 "change this into subtraction"이라고 고침)

수업 메모 02주-2

어떤 연산도 메모리의 데이터에 직접 하지 않는다: 레지스터로 load → 연산 → store. 결과도 레지스터에만 쓸 수 있다. 슬라이드의 마지막 add는 sub 오타.

메모리 피연산자 · 엔디안 · lw/sw

교안 범위: week3 p.10–14

§ 메모리 기본

- 메인 메모리는 배열·구조체·동적 데이터 같은 **composite data**용. 레지스터가 적어서 모든 변수를 담을 수 없음.
- 산술 연산 전에 **load**(메모리 → 레지스터), 끝나면 **store**(레지스터 → 메모리).
- **Byte addressed**: 주소 하나 = 8-bit byte 하나.
- **Words are aligned**: word(32-bit)의 주소는 **4의 배수**.
- 배열 원소 주소 = base address + index × 4 (int 배열).

> **수업 메모** 02주-2
> 메모리는 정렬돼 있으니 배열처럼 생각하면 된다: 원하는 주소 = base + index × 4.

§ Big Endian vs Little Endian

- **Big endian**: MSB(가장 큰 자리 byte)를 **가장 낮은 주소**에.
- **Little endian**: LSB(가장 작은 자리 byte)를 **가장 낮은 주소**에.

0x12345678을 주소 0–3에:

주소	0	1	2	3
Big endian	12	34	56	78
Little endian	78	56	34	12

함정: 16진수 두 자리가 **1 byte**. Little endian에서도 byte 안의 두 자리 순서는 뒤집지 않는다 (78을 87로 쓰면 틀림). 교수님이 두 번 강조.

> **수업 메모** 02주-2
> 16진수 두 자리가 한 숫자(byte). little endian이어도 **두 자리 순서는 절대 바꾸지 말 것** (78을 87로 쓰면 틀림).

§ lw / sw

`lw dest, offset(base)` · `sw src, offset(base)`

- 피연산자 3개: 레지스터, **offset(상수, byte 단위)**, base address(레지스터).
- sw는 첫 번째 레지스터가 **source** (메모리에 쓸 값).

슬라이드 예제 1: `g = h + A[8];` (g:$s1, h:$s2, A의 base:$s3)

```
lw  $t0, 32($s3)      # offset = 8 × 4 = 32
add $s1, $s2, $t0
```

슬라이드 예제 2: `A[12] = h + A[8];`

```
lw  $t0, 32($s3)
add $t0, $s2, $t0
sw  $t0, 48($s3)      # 12 × 4 = 48
```

함정: **offset은 반드시 상수**. 레지스터를 offset 자리에 쓰면 정답이어도 **0점**, OX 문제면 무조건 False. 인덱스가 변수면 주소를 먼저 계산(`sll`, `add`)한 뒤 `lw $t0, 0($t1)`.

수업 메모 02주-2 · 03주-1

lw는 첫 변수가 destination, sw는 첫 변수가 **source**. offset = index × word 크기(4).

§ Registers vs Memory

- 레지스터가 메모리보다 훨씬 빠르고 쓰기 쉬움.
- 메모리 데이터로 연산하려면 load/store가 필요 → 실행 명령어 수 증가.
- 컴파일러는 변수를 최대한 레지스터에 두고, 덜 쓰는 변수만 메모리로 **spill**. 레지스터 최적화가 효율의 핵심.

상수 · 2의 보수 · 부호 확장 · 16진수

교안 범위: week3 p.15–23

§ Immediate 피연산자와 $zero

`addi $s3, $s3, 4` — 명령어 안에 상수를 직접 넣음.

- **subtract immediate는 없다** → 음수 상수로 addi: `addi $s2, $s1, -1`.
- **Design Principle 3: Make the common case fast** — 작은 상수가 흔하므로, immediate가 load를 없애 준다.
- **$zero (레지스터 0)** = 항상 상수 0, **덮어쓸 수 없음**.
- 레지스터 복사(move): `add $t2, $s1, $zero`

함정: 복사는 `add` + `$zero`. `addi`에 `$zero`(레지스터)를 상수 자리처럼 넣으면 0점 (교수님 4주-2). `$zero`는 상수가 아니라 레지스터.

> **수업 메모** 03주-1 · 04주-2
> subtract immediate는 없고 음수 상수로 addi. $zero는 덮어쓸 수 없다. 레지스터 복사는 add + $zero이고, 'you should never use the addition immediate when adding zero ... you will get zero points'.

§ Unsigned와 2의 보수

- **Unsigned** n-bit: `x = x_{n-1}2^{n-1} + … + x_0 2^0`, 범위 **0 ~ $2^n - 1$** (32-bit 최대 4,294,967,295).
- **2's complement** n-bit: `x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + … + x_0 2^0`, 범위 **`-2^{n-1} ~ 2^{n-1} - 1`**.
  - bit 31 = sign bit (1 음수, 0 음이 아님). 음이 아닌 수는 unsigned와 표현이 같음.
  - `1111…1111` = −1, `1000…0000` = −2,147,483,648, `0111…1111` = 2,147,483,647.

> **수업 메모** 03주-1
> 첫 비트가 부호: 0이면 양수, 1이면 음수. x와 $\bar{x}$를 더하면 모든 비트가 1 = −1이라서 −x = $\bar{x}$ + 1.

§ 부호 반전: Complement and add 1

$x + \bar{x} = 1111\ldots1111_2 = -1$ 이므로 $\bar{x} + 1 = -x$.

예: +2 = `0000 … 0010` → 반전 `1111 … 1101` → +1 → `1111 … 1110` = −2.

수업 예제: −10 (32-bit)

```
+10 = 0000 0000 0000 0000 0000 0000 0000 1010
반전 = 1111 1111 1111 1111 1111 1111 1111 0101
 +1  = 1111 1111 1111 1111 1111 1111 1111 0110   (= 0xFFFFFFF6)
```

> **수업 메모** 03주-1
> −10 예제: +10을 32비트로 쓰고 → 0과 1을 전부 뒤집고 → 1을 더한다. 결과가 1로 시작하니 음수임을 확인.

§ 부호 확장 (Sign extension)

더 많은 비트로 표현할 때 **값을 보존**: **sign bit를 왼쪽으로 복제**.

- MIPS에서 쓰는 곳: `addi`(immediate 확장), `lb`·`lh`(로드한 byte/halfword), `beq`·`bne` **(변위 확장)**.
- 8-bit → 16-bit: +2 = `0000 0010` → `0000 0000 0000 0010`, −2 = `1111 1110` → `1111 1111 1111 1110`.

함정: 교안 p.21에는 −2의 확장 결과가 `1111 1111 1000 0010`으로 적혀 있는데 **오타**. 올바른 값은 `1111 1111 1111 1110`.

> **수업 메모** `03주-1`
> 작은 비트를 큰 비트로 늘릴 때 위쪽을 부호로 채운다. '학기 후반에 더 중요해진다'고 예고 → 6주 데이터패스(lw offset, beq)에서 실제로 사용.

---

§ 16진수

16진수 한 자리 = 4 bit. 표: 0=0000 … 9=1001, a=1010, b=1011, c=1100, d=1101, e=1110, f=1111.

예: `eca8 6420` → `1110 1100 1010 1000 0110 0100 0010 0000`

명령어 인코딩: R-format · I-format

교안 범위: week3 p.22–28

§ 명령어 표현의 기본

- 명령어는 **32-bit 이진수(machine code)**로 인코딩. 형식 수를 적게 → **Regularity**.
- **opcode**: 앞 6 bit. 어떤 형식의 명령어인지 알려 줘서 나머지를 해석할 준비를 시킴.
- 레지스터 번호: `$t0-$t7` = 8–15, `$s0-$s7` = 16–23, `$t8-$t9` = 24–25 (그 외: $zero 0, $v0–$v1 2–3, $a0–$a3 4–7, $gp 28, $sp 29, $fp 30, $ra 31).

§ R-format

op	rs	rt	rd	shamt	funct
6 bits	5	5	5	5	6 bits

- **op = 항상 0** (연산 종류는 funct가 결정).
- rs: 첫 번째 source, rt: 두 번째 source, **rd: destination**, shamt: shift amount (지금은 0), funct: function code (opcode 확장).
- funct 값은 시험에서 **주어짐** (외울 필요 없음). 참고: add 32, sub 34, and 36, or 37, slt 42.

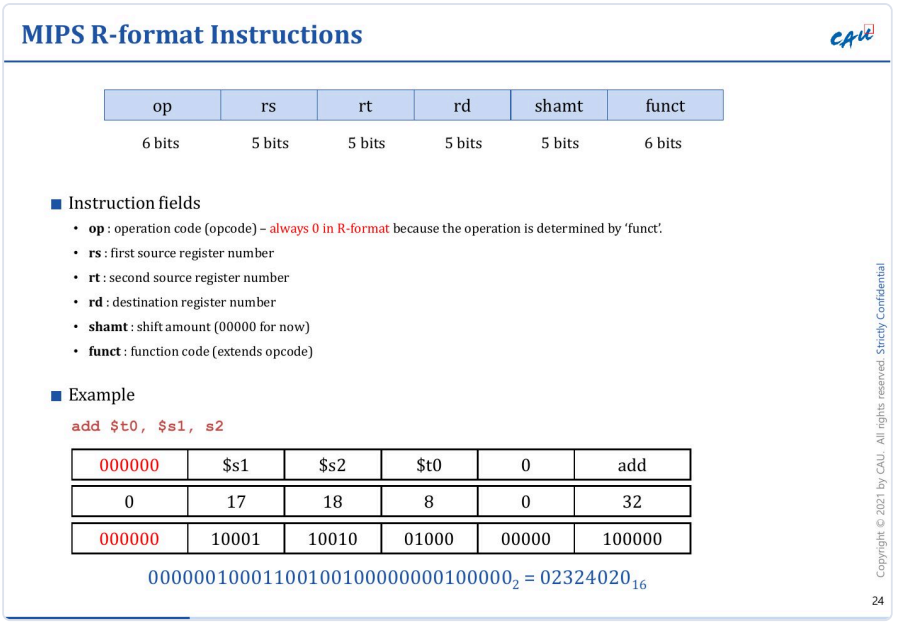
예: `add $t0, $s1, $s2`

```
op      rs     rt     rd     shamt  funct
000000  10001  10010  01000  00000  100000
0       17     18     8      0      32
= 0000 0010 0011 0010 0100 0000 0010 0000₂ = 0x02324020
```

함정: 어셈블리는 **dest, src, src** 순서, 기계어는 **rs, rt, rd** (source 먼저, destination 나중). 교수님이 3주·6주 두 번 다 강조.

수업 메모 03주-1

'You should mind that the order is twisted' — 어셈블리는 dest 먼저, 기계어는 rs, rt, rd. funct 값은 시험에서 주어지니 외울 필요 없다.



week3 p.25 · R-format 필드와 add $t0,$s1,$s2 인코딩

§ I-format

op	rs	rt	constant or address
6 bits	5	5	16 bits

- immediate 산술과 load/store. **rt = destination(lw, addi) 또는 source(sw)**.
- constant: -2^{15} ~ $+2^{15} - 1$. address: rs의 base address에 더해지는 offset.
- 예: addi, lw, sw, beq, bne, slti, andi, ori, lui. opcode 참고: lw 35, sw 43, beq 4, bne 5, addi 8.
- **Design Principle 4: Good design demands good compromises** — 형식이 여러 개면 해석이 복잡하지만 모든 명령어를 32-bit로 통일할 수 있음. 형식은 최대한 비슷하게.

> **수업 메모** 03주-1
> I-format은 rd·shamt가 없어서 상수에 16비트를 쓸 수 있다. 그래서 32비트와 맞추는 sign extension이 필요.

§ 인코딩 예제: A[300] = h + A[300]

`$t1` = A의 base, `$s2` = h.

```
lw  $t0, 1200($t1)     # 300 × 4 = 1200
add $t0, $s2, $t0
sw  $t0, 1200($t1)
```

명령어	op	rs	rt	rd	shamt	funct / address
lw	35	9	8			1200
add	0	18	8	8	0	32
sw	43	9	8			1200

이진: lw = `100011 01001 01000 0000 0100 1011 0000`, sw는 op만 `101011`.

수업 메모 03주-1

A[300] 예제: 300 × 4 = 1200. lw opcode 35, sw opcode 43(10진수). 이렇게 어셈블리가 기계어로 인코딩된다.

논리 연산 · 조건 분기 · if/while

교안 범위: week3 p.30–40 · week4 p.2

§ Shift 연산

연산	C	Java	MIPS		
Shift left	<<	<<	**sll**		
Shift right	>>	>>>	**srl**		
Bitwise AND/OR/NOT	& |	~	& |	~	and, andi / or, ori / **nor**

- R-format의 **shamt**에 몇 칸 밀지 들어감.
- **sll i** = 2^i 곱하기 (오른쪽을 0으로 채움). **srl i** = 2^i 나누기, **unsigned만**.

함정: srl은 왼쪽을 0으로 채우므로 음수(2의 보수)에 쓰면 부호가 깨진다 (교수님: "you should not just perform this operation on a signed number").

수업 메모 03주-1
오른쪽 shift는 0으로 채우니까 부호 있는 수(2의 보수)에 쓰면 부호가 깨진다.

§ AND · OR · NOR

- **AND**: 마스킹. 원하는 비트만 남기고 나머지 0.
- **OR**: 원하는 비트를 1로, 나머지 유지.
- **NOT**은 MIPS에 없음 → **NOR 3-operand**: `nor $t0, $t1, $zero` = NOT $t1.

예 ($t1 = 0…0011 1100 0000 0000, $t2 = 0…0000 1101 1100 0000):

- `and $t0,$t1,$t2` → 0…0000 1100 0000 0000
- `or $t0,$t1,$t2` → 0…0011 1101 1100 0000

§ 분기 명령어

- `beq rs, rt, L1`: rs == rt면 L1로.
- `bne rs, rt, L1`: rs != rt면 L1로.
- `j L1`: 무조건 L1로 점프.

§ if-else 컴파일

`if (i == j) f = g + h; else f = g - h;` (f, g, h, i, j = $s0–$s4)

```
      bne $s3, $s4, Else    # i ≠ j 이면 else로
      add $s0, $s1, $s2     # f = g + h
      j   Exit
Else: sub $s0, $s1, $s2     # f = g - h
Exit:
```

조건을 **반대로** 검사해서 else로 건너뛰는 게 요령 (i == j면 bne가 안 걸려서 if 블록 실행).

수업 메모 04주-1
i == j면 bne가 안 걸려서 if 블록 수행 후 무조건 j Exit. bne가 걸리면(i ≠ j) else 블록의 sub.

§ while 루프 컴파일

`while (save[i] == k) i += 1;` (i: $s3, k: $s5, save의 base: $s6)

```
Loop: sll  $t1, $s3, 2      # t1 = i * 4
      add  $t1, $t1, $s6    # t1 = &save[i]
      lw   $t0, 0($t1)      # t0 = save[i]
      bne  $t0, $s5, Exit   # save[i] ≠ k 이면 탈출
      addi $s3, $s3, 1      # i += 1
      j    Loop
Exit:
```

함정: 인덱스가 변수(i)라서 `lw $t0, $t1($s6)` 처럼 쓰고 싶어도 **안 됨**. 주소를 먼저 계산하고 offset은 0. OX에서 "offset에 레지스터" = False.

수업 메모 04주-1
'if you face a true or false question where the offset is provided as a register, then you must put false' — 그래서 sll·add로 주소를 먼저 계산하고 lw의 offset은 0.

§ slt / slti / sltu

- `slt rd, rs, rt`: rs < rt면 rd = 1, 아니면 0.
- `slti rt, rs, constant`: rs < constant면 rt = 1.
- beq/bne와 조합해서 모든 비교(==, !=, <, <=, >, >=)를 만든다.

```
slt $t0, $s1, $s2       # $s1 < $s2 면 $t0 = 1
bne $t0, $zero, L       # 1 ≠ 0 이면 L로
```

- **Signed vs Unsigned**: slt/slti = signed, sltu/sltiu = unsigned.
  $s0 = 1111...1111, $s1 = 0000...0001일 때 `slt` → −1 < +1 → $t0 = 1, `sltu` → 4,294,967,295 > 1 → $t0 = 0.

수업 메모 04주-1
slt/slti는 뒤의 값이 더 크면 1, 아니면 0.

§ 왜 blt, bge가 없나

- 하드웨어에서 <, > 비교는 =, ≠보다 느림.
- 분기와 합치면 명령어당 일이 많아져 **클럭이 느려짐 → 모든 명령어가 손해**.
- 좋은 절충: **beq, bne** + slt 조합. (`blt`는 assembler의 pseudo-instruction으로 `slt` + `bne`로 바뀜.)

프로시저 호출 · 스택

교안 범위: week3 p.42–51 · week4 p.2–5

§ 프로시저 호출 단계 · jal / jr

1. 인자를 레지스터에 둔다 → 2. 프로시저로 제어 이동 → 3. 저장 공간 확보 → 4. 연산 수행 → 5. 결과를 레지스터에 → 6. 호출한 곳으로 복귀.

- `jal ProcedureLabel` (jump-and-link): **다음 명령어 주소(PC+4)를 $ra에 저장**하고 타깃으로 점프.
- `jr $ra` (jump register): $ra를 PC에 복사 → 복귀. 계산된 점프에도 사용.
- PC: 현재 실행 중인 명령어의 주소를 담은 레지스터.

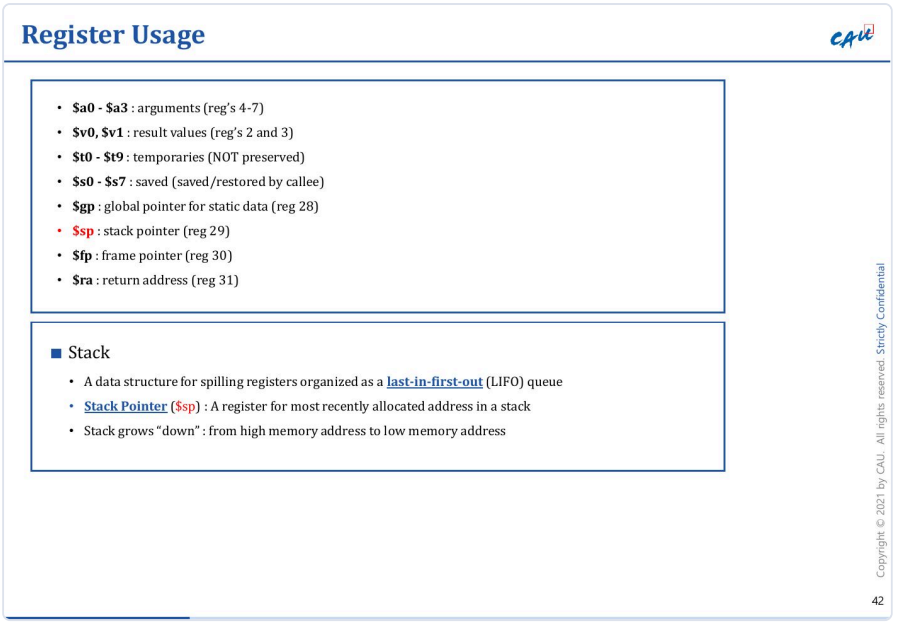
예: 100번지에서 `jal`, 함수는 388번지 → $ra = 104, PC = 388. 함수 끝의 `jr $ra`로 104로 복귀.

> **수업 메모** 04주-1
> 100번 줄에서 jal → $ra = 104(다음 줄), PC = 함수가 정의된 줄(예: 388). 함수의 jr $ra로 104로 돌아온다.

§ 레지스터 사용 규약과 스택

레지스터	번호	용도
$a0–$a3	4–7	인자
$v0, $v1	2–3	결과값
$t0–$t9	8–15, 24–25	임시 (**보존 안 됨**)
$s0–$s7	16–23	saved (**callee가 저장·복원**)
$gp / $sp / $fp / $ra	28 / 29 / 30 / 31	global ptr / **stack ptr** / frame ptr / **return address**

- **Stack**: 레지스터를 spill하는 LIFO 구조. **$sp**는 가장 최근에 할당한 주소.
- **스택은 아래로 자란다** (높은 주소 → 낮은 주소): 공간 확보 = `addi $sp, $sp, -N`.
- Procedure frame (activation record): callee가 할당한 지역 데이터.



week3 p.43 · Register Usage와 스택

§ Leaf procedure 예제

다른 함수를 부르지 않는 함수. `int leaf_example(int g, h, i, j) { int f; f = (g+h) - (i+j); return f; }` (인자 $a0–$a3, f는 $s0)

```
leaf_example:
  addi $sp, $sp, -4      # int 하나(4 byte) 공간
  sw   $s0, 0($sp)       # 원래 $s0 저장
  add  $t0, $a0, $a1     # g + h
  add  $t1, $a2, $a3     # i + j
  sub  $s0, $t0, $t1     # f
  add  $v0, $s0, $zero   # 결과를 $v0로 복사
  lw   $s0, 0($sp)       # $s0 복원
  addi $sp, $sp, 4
  jr   $ra
```

수업 메모 04주-1 · 04주-2

$s0이 다른 곳에서 쓰던 레지스터일 수 있으니 스택에 저장했다가 나갈 때 복원. int 하나라 $sp를 4만큼. 결과를 $v0로 옮길 땐 add + $zero.

§ Non-leaf (재귀) 예제: fact

다른 함수를 부르는 함수는 호출 전에 **$ra와 이후에 필요한 인자**를 스택에 저장, 호출 후 복원.

```
fact:
  addi $sp, $sp, -8      # 2칸: 인자, 복귀주소
  sw   $ra, 4($sp)
  sw   $a0, 0($sp)
  slti $t0, $a0, 1       # n < 1 ?
  beq  $t0, $zero, L1    # 아니면 L1
  addi $v0, $zero, 1     # return 1
  addi $sp, $sp, 8
  jr   $ra
L1:
  addi $a0, $a0, -1      # n - 1
  jal  fact
  lw   $a0, 0($sp)       # 저장한 그 자리에서 복원
  lw   $ra, 4($sp)
  addi $sp, $sp, 8
  mul  $v0, $a0, $v0     # n * fact(n-1)
  jr   $ra
```

함정: **저장한 offset과 복원하는 offset이 같아야** 한다. 인자를 0에, $ra를 4에 넣었으면 복원도 그대로. 반대로 꺼내면 0점 (교수님 4주-1).

함정: 교안 week3 슬라이드의 C 코드는 `return f`로 오타. week5 슬라이드에서 `return 1`로 고쳐짐.

수업 메모 04주-1
인자를 offset 0, $ra를 offset 4에 저장했으면 복원도 같은 자리에서. 'if you ... load this on the opposite order, then you will get a zero point in your midterm'.

§ fact(3) 스택 추적

호출한 곳: 100번지 `jal fact` → $ra = 104, $a0 = 3. $sp 초기값 500.

단계	$sp	0($sp)에 저장	4($sp)에 저장
fact(3) 진입	492	a0 = 3	ra = 104
fact(2) 진입	484	a0 = 2	ra = L1+8
fact(1) 진입	476	a0 = 1	ra = L1+8
fact(0) 진입	468	a0 = 0	ra = L1+8

fact(0): slti → 0 < 1 이므로 $t0 = 1 → beq 안 걸림 → $v0 = 1, $sp = 476, jr → L1+8.
되돌아오며 `lw`로 a0을 1, 2, 3으로 복원하고 $v0 = 1 → 1 → 2 → 6. 마지막 jr은 **104**로.

함정: `slti $t0, $a0, 1`에서 a0 = 1이면 1 < 1이 거짓 → $t0 = 0 (재귀 계속). 교수님이 "be extremely careful"이라고 짚은 곳.

수업 메모 04주-1
fact(3)을 $sp 500에서 시작해 한 줄씩 추적: 492 → 484 → 476 → 468. slti에서 1 < 1은 거짓이라 한 번 더 재귀 — 'be extremely careful'.

§ 인자 타입별 스택 크기 (교수님 기준)

- int 인자 1개 + $ra → 4 + 4 = **8 byte**.
- **교수님 설명**: char 인자는 1 byte씩. 예) char 인자 2개 + $ra → 4 + 1 + 1 = **6 byte**, char 3개면 4 + 3 = 7.
- 저장하는 값이 int면 char 배열을 다뤄도 4씩 움직인다 (strcpy에서 저장하는 건 int i).

주의: 실제 MIPS 컴파일러는 정렬 때문에 보통 4의 배수로 잡지만, 수업에서는 위처럼 설명했어. 시험에선 수업 기준으로.

수업 메모 04주-2 · 05주-1

인자 타입을 꼭 보라: char 인자는 1 byte씩이라 char 2개 + $ra = 6 byte, char 3개면 7 byte. 'when actually facing your midterm, be especially careful about this format'.

---

§ Byte 연산 · strcpy

- `lb rt, offset(rs)` / `lh`: sign extend해서 32-bit로. `lbu` / `lhu`: zero extend.
- `sb` / `sh`: 가장 오른쪽 byte/halfword만 저장.

`void strcpy(char x[], char y[])` (x, y 주소는 $a0, $a1, i는 $s0)

```
strcpy:
  addi $sp, $sp, -4
  sw   $s0, 0($sp)
  add  $s0, $zero, $zero   # i = 0
L1: add  $t1, $s0, $a1     # &y[i]  (char라서 ×4 없음)
  lbu  $t2, 0($t1)
  add  $t3, $s0, $a0       # &x[i]
  sb   $t2, 0($t3)
  beq  $t2, $zero, L2      # '\0' 이면 끝
  addi $s0, $s0, 1
  j    L1
L2: lw   $s0, 0($sp)
  addi $sp, $sp, 4
  jr   $ra
```

함정: char 배열은 원소가 1 byte라서 **sll로 ×4 하지 않는다**. int 배열과 다른 점.

수업 메모 04주-2

char 배열은 인덱스에 ×4(sll)를 하지 않는다. lbu로 1 byte만 읽는다. lbu도 I-type이라 offset은 상수 — 레지스터를 넣으면 '정답이든 아니든 0점'.

주소 지정 · 32-bit 상수 · 번역과 링크

교안 범위: week3 p.49–67

§ 32-bit 상수: lui + ori

대부분 상수는 16-bit immediate로 충분. 가끔 32-bit 상수가 필요하면:

- `lui rt, constant`: 16-bit 상수를 rt의 **위 16 bit**에, 아래 16 bit는 0.
- 아래 16 bit는 **ori**로 채움 (addi 쓰지 말 것: 부호 확장 때문).

```
lui $s0, 61          # 0000 0000 0111 1101 0000 0000 0000 0000
ori $s0, $s0, 2304   # 0000 0000 0111 1101 0000 1001 0000 0000
```

§ Branch addressing (PC-relative)

op	rs	rt	constant or address (16)

- beq/bne는 I-format. 대부분 분기 대상은 가까움(루프, if).
- **Target address = (PC + 4) + offset × 4**
- offset은 **명령어(word) 개수 차이**, 부호 있음 → **±2^{15} words** (= ±2^{17} bytes) 범위.
- 너무 멀면 assembler가 조건을 뒤집고 j로 우회하게 다시 씀:

```
beq $s0, $s1, L1      →      bne $s0, $s1, L2
                             j   L1
                        L2: …
```

시험 포인트: 6주에 교수님이 판서로 다시 설명: offset = (target − (PC + 4)) / 4. 예) PC 108, target 124 → (124 − 112)/4 = **3**. (4주-2 미리보기 설명은 대략적이었고 6주 설명이 정확한 규칙.)

수업 메모 04주-2
분기 대상이 16비트로 표현 못 할 만큼 멀면 중간 지점을 두고 두 번 뛴다. 자세한 건 기말에. (정확한 오프셋 규칙은 6주 판서 참고)

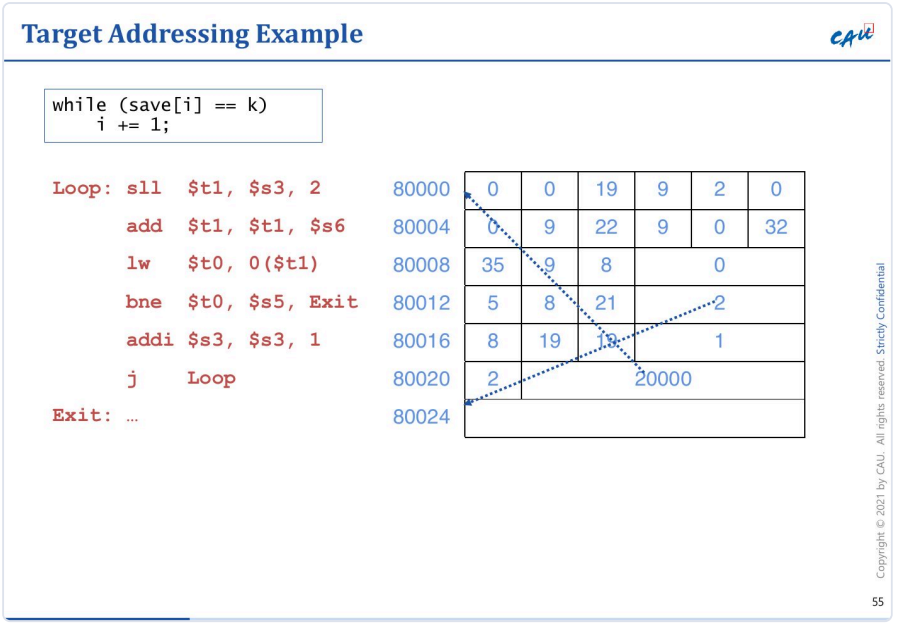
§ Jump addressing (J-format)

op (6)	address (26)

- j, jal: 대상이 text segment 어디든 될 수 있음 → 절대 주소를 인코딩.
- (Pseudo) direct jump: **Target = address × 4** (위 4 bit는 PC에서). 2^{26} words = 2^{28} bytes 범위.
- `jr`: 32-bit 전체 주소가 레지스터에 → R-format.

슬라이드 예제 (Loop가 80000번지):

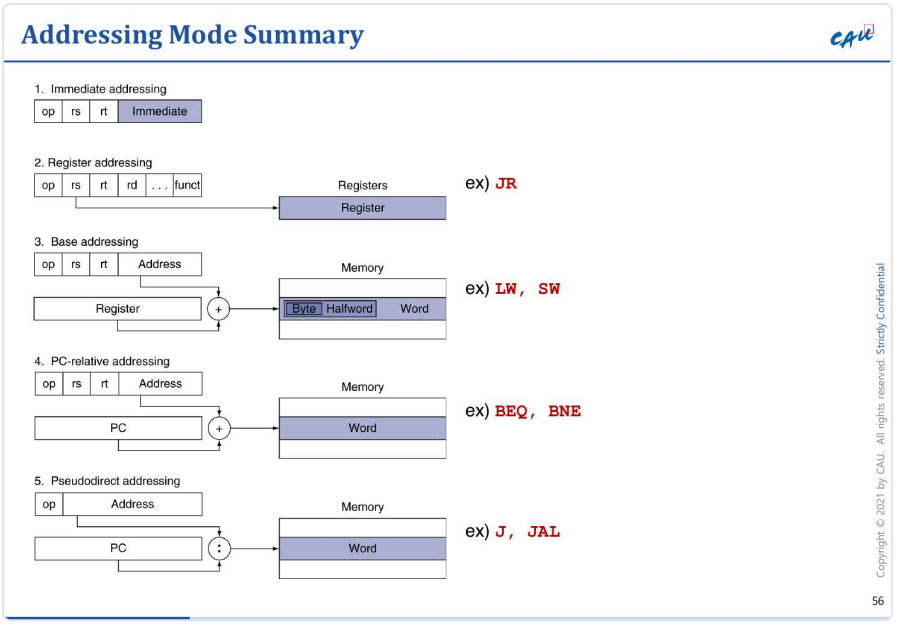
```
80000 Loop: sll  $t1,$s3,2     0  0 19  9 2  0
80004       add  $t1,$t1,$s6   0  9 22  9 0 32
80008       lw   $t0,0($t1)   35  9  8   0
80012       bne  $t0,$s5,Exit  5  8 21   2      → (80012+4) + 2×4 = 80024 = Exit
80016       addi $s3,$s3,1     8 19 19   1
80020       j    Loop          2   20000        → 20000 × 4 = 80000 = Loop
80024 Exit:
```



week3 p.56 · Target Addressing Example (80000번지 루프)

§ Addressing modes 요약

1. Immediate (addi), 2. Register (add, jr), 3. Base/displacement (**lw, sw**), 4. PC-relative (**beq, bne**), 5. Pseudo-direct (**j, jal**).



week3 p.57 · Addressing Mode Summary

§ Pseudo-instruction · 번역 · 링크

- **Pseudo-instruction**: 어셈블러가 진짜 명령어로 바꿔 주는 가짜 명령어. `move $t0,$t1` → `add $t0,$zero,$t1`, `blt $t0,$t1,L` → `slt $at,$t0,$t1` + `bne $at,$zero,L`.
- 오브젝트 모듈: header, text segment, static data, relocation info, symbol table, debug info.
- **Linker**: 세그먼트 병합 → 레이블 주소 결정 → 위치 의존·외부 참조 수정 → executable.
- **Loader**: header 읽기 → 가상 주소 공간 생성 → text·data 복사 → 스택에 인자 → 레지스터 초기화($sp, $fp, $gp) → startup routine으로 점프.
- **Dynamic linking**: 호출될 때만 라이브러리를 링크/로드. relocatable 코드 필요, 이미지 비대 방지, 새 버전 자동 반영.
- **Memory layout**: Text(코드) → Static data(전역, $gp) → Dynamic data(heap, malloc) → Stack(자동 저장).

---

§ Fallacies와 설계 원칙 정리

- Fallacy: 강력한 명령어 = 높은 성능 (복잡한 명령어는 구현이 어려워 단순한 것까지 느려짐).
- Fallacy: 고성능을 위해 어셈블리로 작성 (현대 컴파일러가 더 잘함).
- x86은 명령어 집합을 바꾸지 않고 계속 추가 (backward compatibility).
- **설계 원칙 4개**: 1. Simplicity favors regularity, 2. Smaller is faster, 3. Make the common case fast, 4. Good design demands good compromises.

U3 · week4-수업.pdf p.6–21 · week5-수업.pdf

Ch.3 컴퓨터 연산 (정수 · 부동소수점)

곱셈·나눗셈은 "회로를 주고 과정을 보여라" 형식. 교수님이 "회로에 작은 변화가 있을 수 있으니 잘 보라"고 했어. 중간엔 product·multiplicand 8 bit, multiplier 4 bit 크기로만 나옴. 부동소수점은 필드 비트 수를 하나라도 틀리면 0점.

덧셈 · 뺄셈 · 오버플로

교안 범위: week4 p.6–13

§ 정수 덧셈과 뺄셈

- 덧셈은 십진수와 같음: 자리마다 더하고 2 이상이면 carry를 다음 자리로.
- 예: 7 + 6 = `0111 + 0110 = 1101` (13).
- **뺄셈 = 음수 더하기**: 7 − 6 = 7 + (−6). −6은 2의 보수(반전 + 1)로 만든다.

§ 오버플로 판정

결과가 표현 범위(예: 32-bit)를 넘으면 overflow.

연산	오버플로 조건
양수 + 음수	**절대 없음**
양수 + 양수	결과의 sign bit가 **1**
음수 + 음수	결과의 sign bit가 **0**
양수 − 양수, 음수 − 음수	없음
음수 − 양수	결과 sign이 **0**
양수 − 음수	결과 sign이 **1**

원리: 2의 보수에서 첫 비트가 부호. 양수 둘을 더해 음수가 나올 수 없으니 sign이 1이면 오버플로. 현대 CPU는 오버플로를 **exception**으로 처리.

> **수업 메모** 04주-2
> 양수 + 음수는 절대 오버플로가 없다. 첫 비트(부호)만 보면 판정된다. 현대 CPU는 오버플로를 exception으로 처리해 회로를 보호.

곱셈 하드웨어

교안 범위: week4 p.14–18 · week5 p.5–8, 14–15

§ 곱셈의 원리

종이 곱셈과 같다: multiplier의 각 자리를 보고 1이면 multiplicand를 (자리만큼 shift해서) 더하고, 0이면 더하지 않음.

```
  multiplicand     1000
× multiplier     × 1001
                   1000
                  0000
                 0000
                1000
  product       1001000      (8 × 9 = 72)
```

n-bit × n-bit → 결과는 최대 **2n bit**.

§ 원래 곱셈기 (first version)

구성: **Multiplicand 64-bit (Shift left)**, **Multiplier 32-bit (Shift right)**, **64-bit ALU**, **Product 64-bit**, Control test. (중간엔 8-bit / 4-bit / 8-bit로 축소)

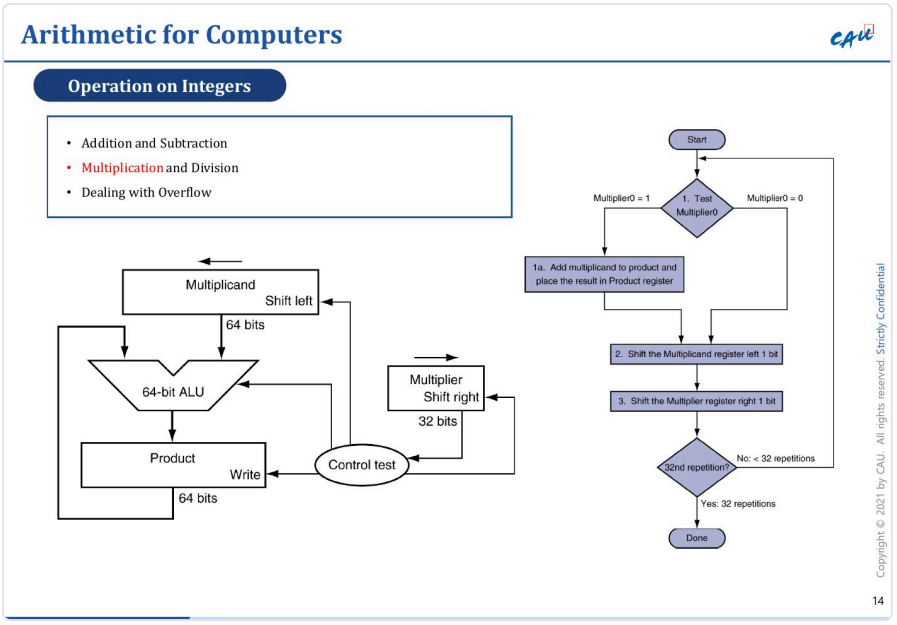
알고리즘:

1. **Product = 0**으로 초기화.
2. Multiplier의 **LSB 검사**: 1이면 `Product = Product + Multiplicand` (ALU에 덧셈 신호), 0이면 아무것도 안 함.
3. 결과와 상관없이 **매 반복**: Multiplicand **왼쪽 shift**, Multiplier **오른쪽 shift**.
4. 32번(중간 크기에선 4번) 반복하면 끝. (교수님 설명: multiplier가 0이 되면 종료)

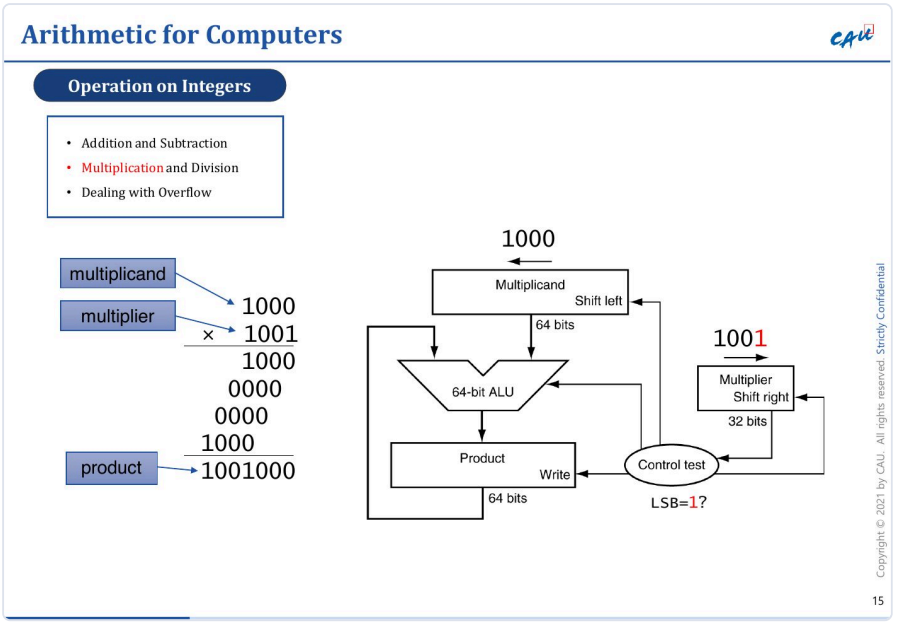
1000 × 1001:

반복	단계	Multiplier	Multiplicand	Product
0	초기값	1001	0000 1000	0000 0000
1	LSB=1 → Prod += Mcand	1001	0000 1000	0000 1000
1	Mcand <<1, Mplier >>1	0100	0001 0000	0000 1000
2	LSB=0 → 덧셈 없음, shift	0010	0010 0000	0000 1000
3	LSB=0 → 덧셈 없음, shift	0001	0100 0000	0000 1000
4	LSB=1 → Prod += Mcand	0001	0100 0000	0100 1000
4	shift	0000	1000 0000	**0100 1000** (= 72)

> **수업 메모** 04주-2 · 05주-1
> 중간에서는 product·multiplicand 8 bit, multiplier 4 bit로만 나온다. LSB 결과와 상관없이 **매 반복 두 번 shift** (multiplier 오른쪽, multiplicand 왼쪽).



week4 p.15 · 원래 곱셈기와 흐름도



week4 p.16 · 1000 × 1001: LSB 검사

§ 개선된 곱셈기 (refined version)

구성: **Multiplicand 32-bit (shift 안 함)**, **32-bit ALU**, **Product 64-bit (Shift right)**. Multiplier 레지스터가 따로 없음.

알고리즘:

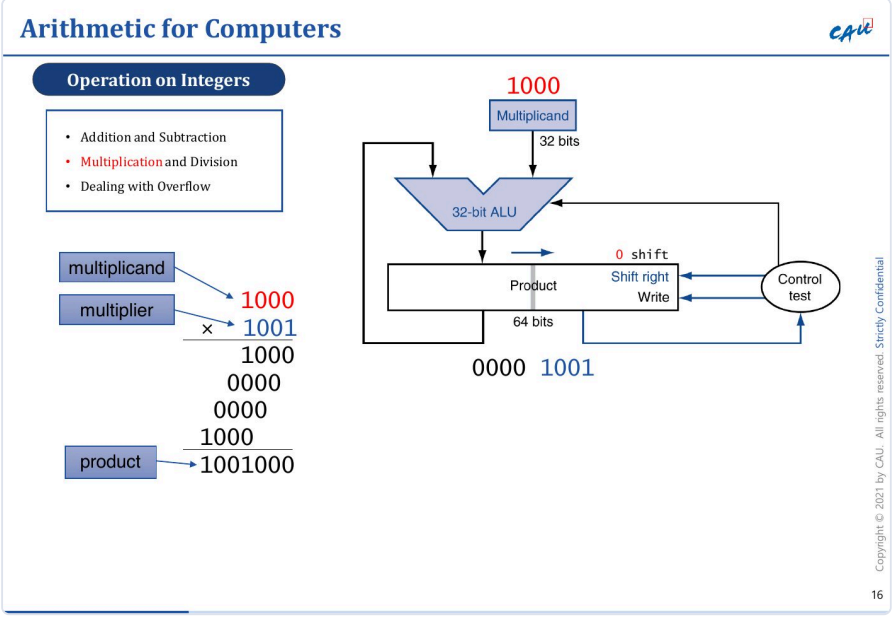
1. Product의 **아래 절반에 multiplier**를 넣고 위 절반은 0.
2. Product의 **LSB 검사**: 1이면 Product의 **위 절반에 multiplicand를 더함**.
3. 매 반복: Product 전체를 **오른쪽 shift**.
4. 종료: **shift 횟수 = multiplier 비트 수** (shift counter).

함정: 종료 조건의 기준은 **multiplier**의 비트 수. multiplicand 아님 (교수님: "be extremely careful it is not the multiplicand").

1000 × 1001:

반복	단계	Product
0	초기값 (아래 = multiplier)	0000 1001
1	LSB=1 → 위 절반 += 1000	1000 1001
1	>> 1	0100 0100
2	LSB=0, >> 1	0010 0010
3	LSB=0, >> 1	0001 0001
4	LSB=1 → 위 절반 += 1000	1001 0001
4	>> 1	**0100 1000** (= 72)

> **수업 메모** `05주-1`
> 'I will actually provide you a circuit and ask you what to do for the operation ... there will be minor tweaks within the circuit'. 종료는 shift 횟수 = **multiplier** 비트 수 (multiplicand 아님).



week4 p.17 · 개선된 곱셈기 (Product shift right)

§ 수업 예제 1·2 (week5)

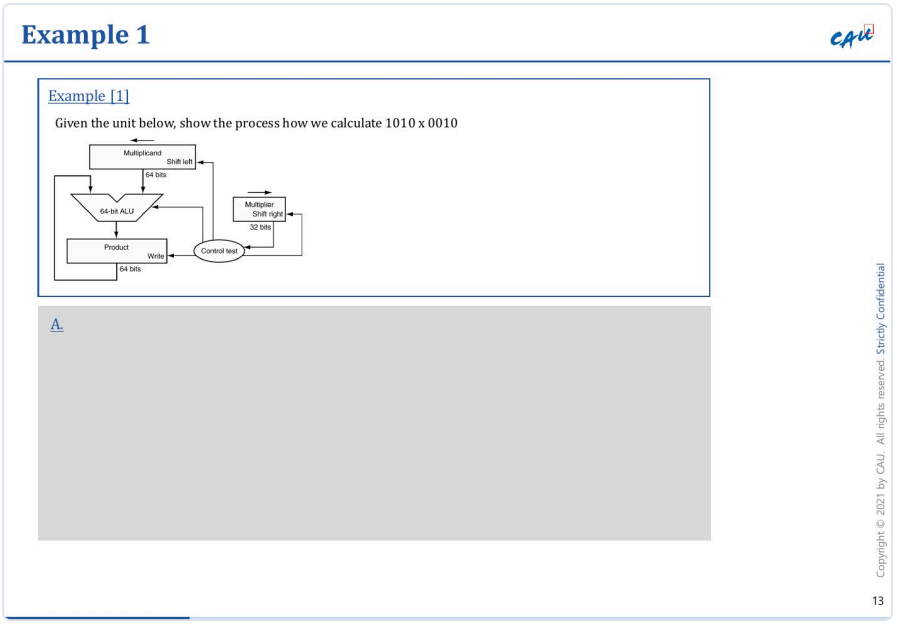
Example 1: 1010 × 0010 (원래 곱셈기) → 10 × 2 = 20

반복	LSB	동작	Multiplier (shift 후)	Multiplicand (shift 후)	Product
0		초기값	0010	0000 1010	0000 0000
1	0	덧셈 없음	0001	0001 0100	0000 0000
2	1	Prod += 0001 0100	0000	0010 1000	0001 0100
3	0	덧셈 없음	0000	0101 0000	0001 0100
4	0	덧셈 없음	0000	1010 0000	**0001 0100** (= 20)

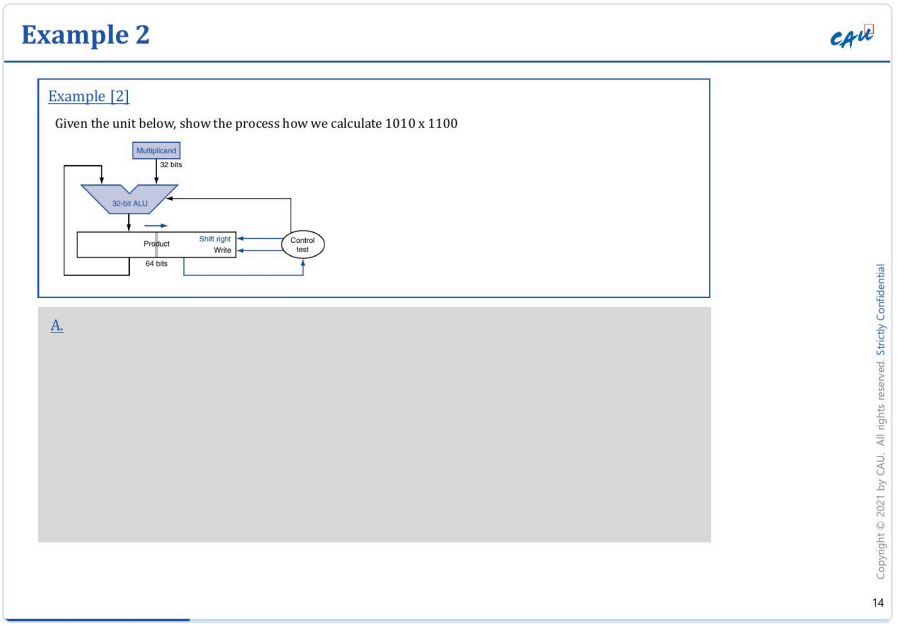
Example 2: 1010 × 1100 (개선 곱셈기) → 10 × 12 = 120

반복	LSB	동작	Product (동작 후)	Product (>>1 후)
0		초기값	0000 1100	
1	0	덧셈 없음	0000 1100	0000 0110
2	0	덧셈 없음	0000 0110	0000 0011
3	1	위 절반 += 1010	1010 0011	0101 0001
4	1	위 절반 += 1010	1111 0001	**0111 1000** (= 120)

수업 메모 `05주-1`
시험 형식 예고: 명령어와 설명을 주고 박스를 채우거나 올바른 MIPS 표현을 쓰는 문제. 곱셈·나눗셈은 회로를 주고 과정을 묻는다.



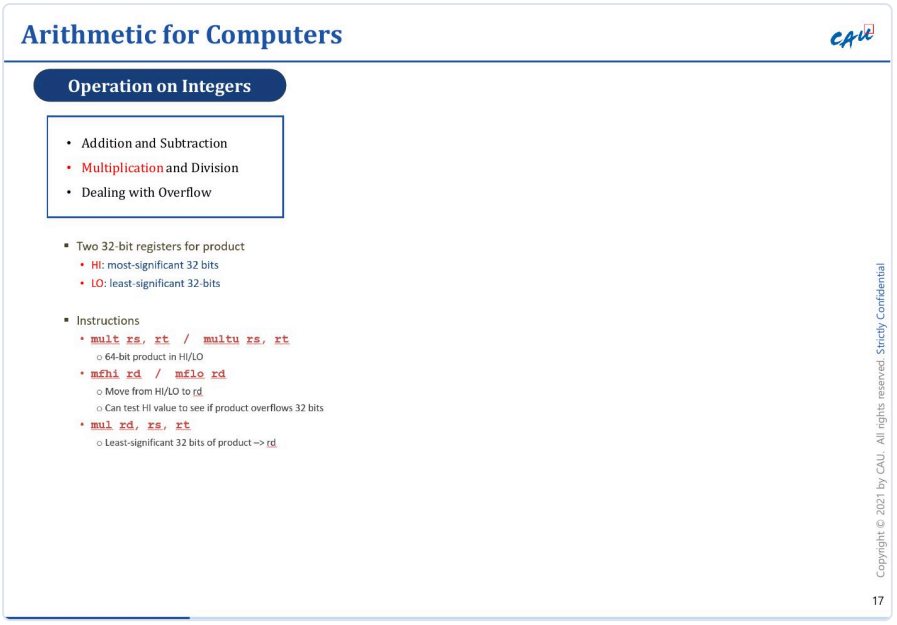
week5 p.14 · Example 1 (원래 곱셈기)



week5 p.15 · Example 2 (개선 곱셈기)

§ MIPS 곱셈 명령어

- 64-bit 곱 결과용 레지스터 2개: **HI** (위 32 bit), **LO** (아래 32 bit).
- `mult rs, rt` / `multu rs, rt` : 64-bit 곱을 HI/LO에.
- `mfhi rd` / `mflo rd` : HI/LO를 rd로 옮김. HI를 보면 32 bit 오버플로 여부를 알 수 있음.
- `mul rd, rs, rt` : 곱의 아래 32 bit만 rd에.



week4 p.18 · HI/LO, mult, mfhi, mflo, mul

나눗셈 하드웨어

교안 범위: week4 p.19–21 · week5 p.9–13, 16

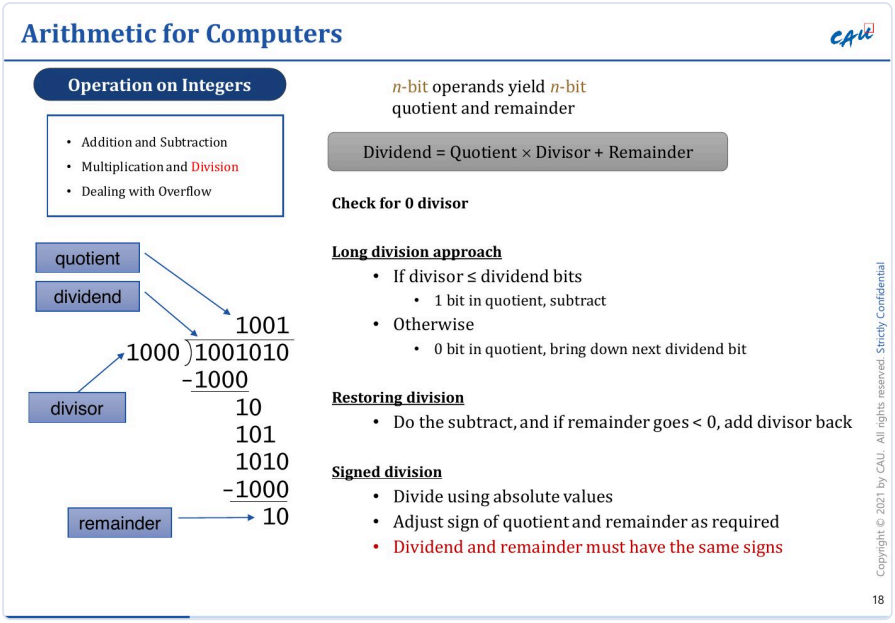
§ 나눗셈의 원리

`Dividend = Quotient × Divisor + Remainder`. n-bit 피연산자 → n-bit 몫과 나머지.

- **0으로 나누는지 먼저 검사** (교안: "Always check for 0 divisor!!").
- Long division: divisor ≤ 지금 보고 있는 dividend 비트들이면 몫에 1 쓰고 빼기, 아니면 몫에 0 쓰고 다음 비트를 내림.
- **Restoring division**: 일단 빼 보고, 나머지가 **음수(<0)면 divisor를 다시 더해 원상 복구**.
- Signed division: 절댓값으로 나누고 몫·나머지 부호를 조정. **Dividend와 remainder는 같은 부호.**

교안 예: 1001010 ÷ 1000 = 몫 **1001**, 나머지 **10** (74 ÷ 8 = 9 … 2).

> **수업 메모** 04주-2 · 05주-1
> 십진 나눗셈 728 ÷ 8로 설명: 먼저 빼 보고 음수면 다시 더해서(plus back) 원상 복구한 뒤 다음 자리로.



week4 p.19 · 나눗셈 원리와 restoring division

§ 나눗셈기 (first version)

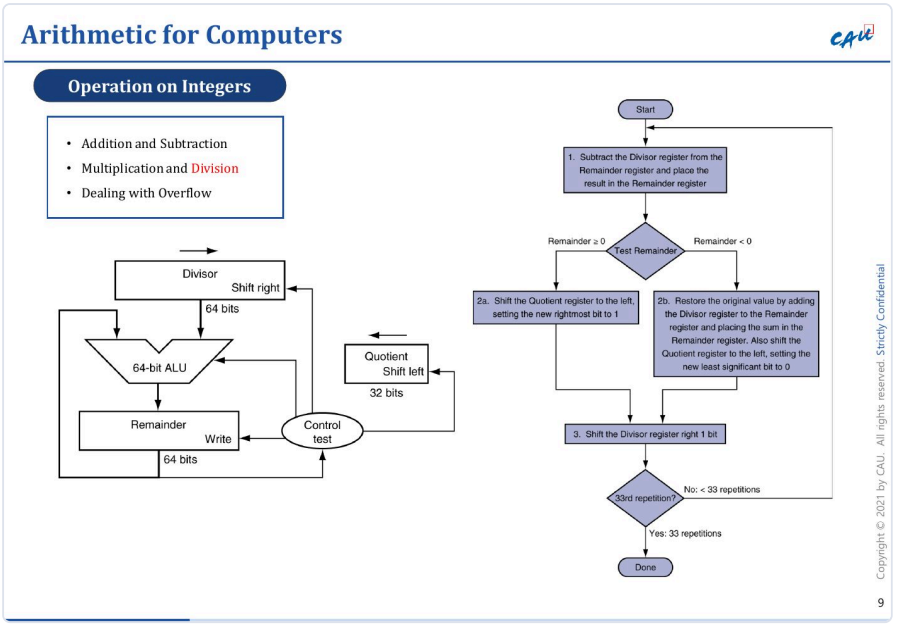
구성: **Divisor 64-bit (Shift right, 처음엔 왼쪽 절반에)**, **64-bit ALU**, **Remainder 64-bit (처음엔 dividend)**, **Quotient 32-bit (Shift left)**.

알고리즘 (n+1번 반복):

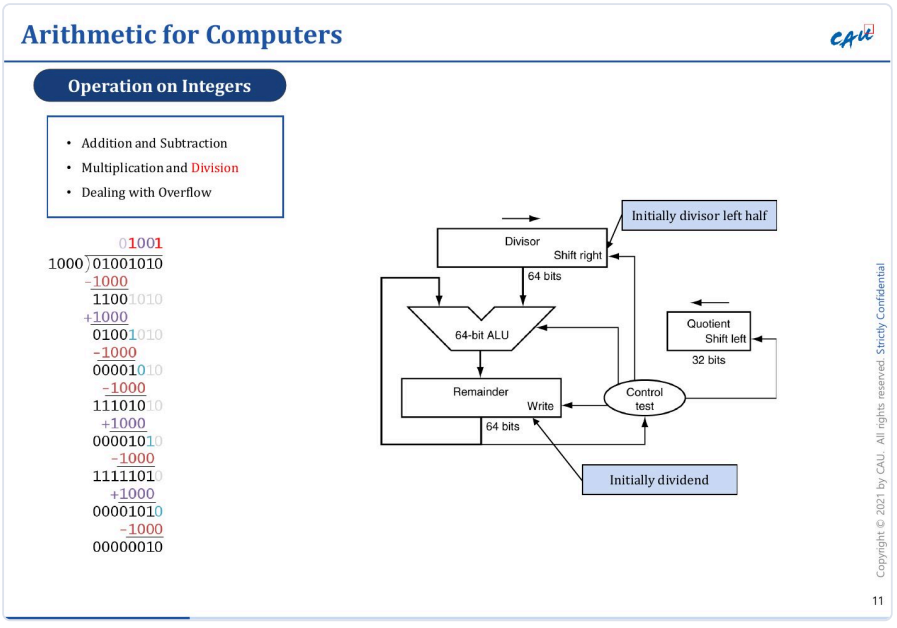
1. `Remainder = Remainder - Divisor`
2. 결과 검사:
   - **≥ 0**: Quotient 왼쪽 shift, 새 LSB = **1**
   - **< 0**: `Remainder = Remainder + Divisor`로 **복구**, Quotient 왼쪽 shift, 새 LSB = **0**
3. Divisor **오른쪽 shift**.

함정: 음수인지는 결과의 **첫 비트(sign bit)**로 판단 (2의 보수). 음수면 반드시 다시 더해서(add back) 원래 값으로 되돌린 뒤 다음 자리로.

> **수업 메모** `05주-1`
> 음수인지는 2의 보수 첫 비트로 안다. 음수면 몫을 붙일 수 없고, 같은 값을 다음 자리와 이어 붙여야 하니 반드시 더해서 원래 값으로 복구.



week5 p.10 · 나눗셈기와 흐름도



week5 p.12 · 01001010 ÷ 1000 빼기·복구 과정

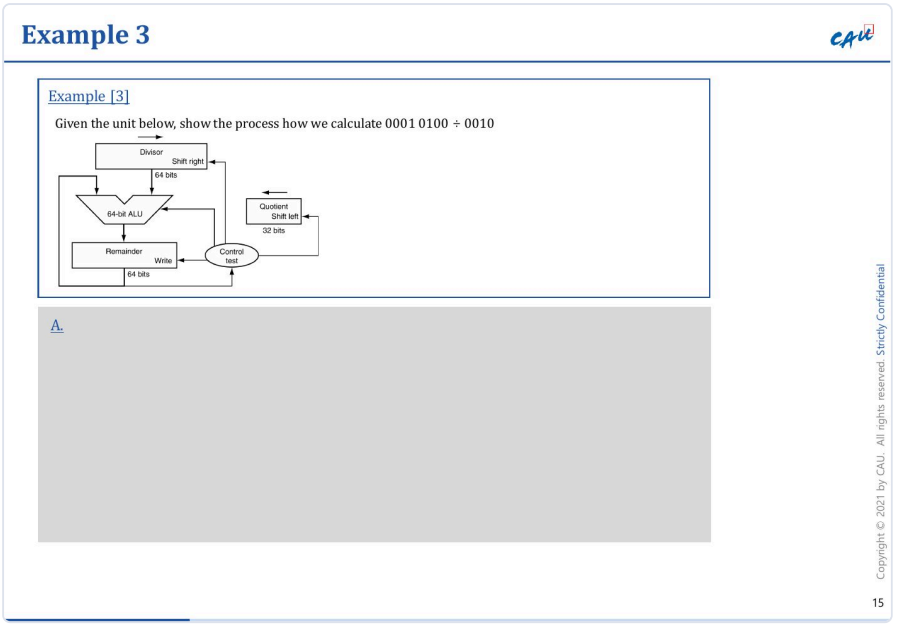
§ 수업 예제 3: 0001 0100 ÷ 0010

20 ÷ 2 = 10 … 0. 4-bit 나눗셈이라 **5번** 반복. Divisor 초기값 = 0010을 왼쪽 절반에 → `0010 0000`.

반복	Rem − Div	판정	Quotient	Divisor (>>1 후)	Remainder
0		초기값	0000	0010 0000	0001 0100

반복	Rem – Div	판정	Quotient	Divisor (>>1 후)	Remainder
1	1111 0100	< 0 → 복구, 0	0000	0001 0000	0001 0100
2	0000 0100	≥ 0 → 1	0001	0000 1000	0000 0100
3	1111 1100	< 0 → 복구, 0	0010	0000 0100	0000 0100
4	0000 0000	≥ 0 → 1	0101	0000 0010	0000 0000
5	1111 1110	< 0 → 복구, 0	**1010**	0000 0001	**0000 0000**

결과: Quotient = 1010 (10), Remainder = 0.



week5 p.16 · Example 3 (나눗셈기)

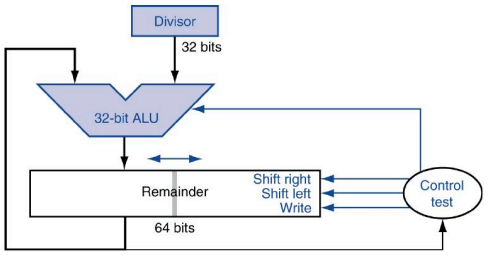
§ 개선된 나눗셈기

교안 week5 p.13: **Divisor 32-bit (shift 안 함)**, **32-bit ALU**, **Remainder 64-bit (Shift right / Shift left / Write)**. 몫이 Remainder 레지스터의 아래 절반에 쌓이고 나머지는 위 절반에 남음 (곱셈 개선판과 같은 아이디어: 레지스터 하나로 합치기).

Arithmetic for Computers

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

week5 p.13 · 개선된 나눗셈기

IEEE 754 부동소수점 표현

교안 범위: week5 p.17–26

§ IEEE 754 형식

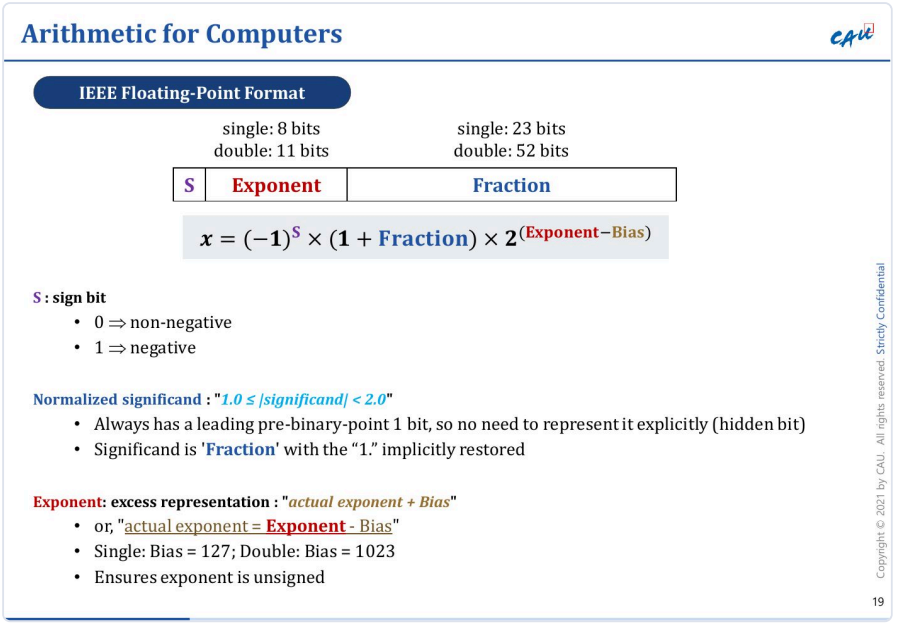
	S	Exponent	Fraction	Bias
Single (float)	1	**8 bits**	**23 bits**	**127**
Double (double)	1	**11 bits**	**52 bits**	**1023**

`x = (-1)^S × (1 + Fraction) × 2^(Exponent - Bias)`

- **S**: 0 = 음이 아님, 1 = 음수.
- **Normalized significand**: 1.0 ≤ |significand| < 2.0. 앞의 "1."은 항상 있으니 **저장하지 않음 (hidden bit)**.
- **Exponent는 excess(bias) 표현**: 저장값 = 실제 지수 + Bias. 그래서 exponent는 항상 unsigned.

함정: 필드 비트 수(1/8/23, 1/11/52)를 하나라도 틀리면 다른 필드까지 밀려서 **전부 0점** (교수님 5주).

수업 메모 05주-1
필드 비트 수를 하나라도 놓치면 필드끼리 섞여서 **0점**. 전체 표현을 다 써야 한다.



week5 p.20 · IEEE 754 형식

§ 변환 예제

−0.75: 0.75 = 0.11_2 = **$1.1_2 \times 2^{-1}$** → S = 1, Exponent = −1 + 127 = 126 = `01111110`, Fraction = `1000…0`

```
1 01111110 10000000000000000000000   = 0xBF400000
```

Double: Exponent = −1 + 1023 = 1022 = `01111111110`, Fraction = `1000…0` (52 bit).

0.625: 0.101_2 = **$1.01_2 \times 2^{-1}$** → S = 0, E = 126, F = `01000…0` → `0 01111110 01000000000000000000000` = 0x3F200000

−3.75: 11.11_2 = **$1.111_2 \times 2^1$** → S = 1, E = 1 + 127 = 128 = `10000000`, F = `111000…0` → `1 10000000 11100000000000000000000` = 0xC0700000

순서: ① 부호 결정 ② 이진 소수로 변환 ③ 1.xxx × 2^e로 정규화 ④ e + bias ⑤ 소수점 뒤 비트를 fraction에 (나머지 0).

§ 범위 · 특수값 · 정밀도

Single에서 exponent `00000000`과 `11111111`은 **예약**:

Exponent	Fraction	의미
0	0	**0**
0	≠ 0	**Denormal** (hidden bit = 0, `(0 + F) × 2^(-bias)`)
255	0	**±∞**
255	≠ 0	**NaN** (0.0/0.0 같은 정의 안 된 결과)

- Single 최소(정규): exponent 1 → 1 − 127 = −126, $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Single 최대: exponent 254 → +127, $\approx \pm 2.0 \times 2^{127} \approx \pm 3.4 \times 10^{38}$
- Double: −1022 ~ +1023, $\approx \pm 2.2 \times 10^{-308} \sim \pm 1.8 \times 10^{308}$
- 정밀도: single $\approx 2^{-23}$ → 약 **6자리** 십진 정밀도, double $\approx 2^{-52}$ → 약 **16자리**.
- Denormal은 정규화 수보다 작은 값을 표현해 **gradual underflow** 허용.

부동소수점 덧셈 · 곱셈 · FP 명령어

교안 범위: week5 p.27–31

§ FP 덧셈 4단계

1. **Align**: 지수가 작은 쪽을 shift해서 큰 지수에 맞춤.
2. **Add significands**.
3. **Normalize** 하고 over/underflow 검사.
4. **Round**, 필요하면 다시 normalize.

십진 예: $9.999\times10^{1} + 1.610\times10^{-1} \rightarrow 9.999\times10^{1} + 0.016\times10^{1} \rightarrow 10.015\times10^{1} \rightarrow 1.0015\times10^{2} \rightarrow \mathbf{1.002\times10^{2}}$

이진 예: $0.5 + (-0.4375) = 1.000_2\times2^{-1} + (-1.110_2\times2^{-2})$

```
1. 1.000×2⁻¹ + (-0.111×2⁻¹)
2. 0.001×2⁻¹
3. 1.000×2⁻⁴  (over/underflow 없음)
4. 1.000×2⁻⁴ = 0.0625
```

§ FP 곱셈 5단계

1. **Add exponents** (biased면 합에서 bias를 한 번 빼기).
2. **Multiply significands**.
3. **Normalize**, over/underflow 검사.
4. **Round**, 필요하면 renormalize.
5. **부호 결정** (피연산자 부호로).

이진 예: $0.5 \times (-0.4375) = 1.000_2\times2^{-1} \times (-1.110_2\times2^{-2})$

```
1. 지수: -1 + -2 = -3   (biased: (-1+127) + (-2+127) - 127 = -3 + 127)
2. 1.000 × 1.110 = 1.110  → 1.110×2⁻³
3. 정규화 그대로, over/underflow 없음
4. 반올림 그대로
5. +ve × -ve → -ve :  -1.110×2⁻³ = -0.21875
```

함정: biased 지수끼리 더하면 bias가 두 번 들어가니까 **127을 한 번 빼야** 한다.

§ FP 하드웨어와 명령어

- 보통 FP 전용 하드웨어(여러 사이클): 덧셈·뺄셈·곱셈·나눗셈·역수·제곱근, FP ↔ 정수 변환.
- **별도의 FP 레지스터**: single 32개 `$f0-$f31`, double은 짝으로 `$f0/$f1`, `$f2/$f3`...
- FP 명령어는 FP 레지스터에서만 동작 (정수 연산을 FP 데이터에 하지 않음) → 코드 크기 영향 적게 레지스터를 늘림.

U4 · week6-수업.pdf

Ch.4 프로세서: 데이터패스와 제어 (단순 버전)

교수님이 6주-2에서 "오늘 목표는 이 슬라이드 한 장(p.3 단순 CPU)을 완벽히 이해하는 것"이라고 함. 명령어마다 어떤 선으로 데이터가 흐르고, 각 MUX에 0/1 중 뭘 줘야 하는지가 핵심. 개념을 읽은 다음 '수업 따라가기' 탭에서 교수님 설명을 장면별로 보고, '데이터패스 놀이터'에서 명령어별 경로를 확인해.

프로세서 개요 · 명령어 실행 단계

교안 범위: week6 p.2–7

§ 두 가지 구현과 명령어 부분집합

- CPU 성능 요소: Instruction Count, **CPI와 Cycle Time** (이 단원은 CPI·cycle time을 결정하는 구현).
- 두 가지 MIPS 구현: **단순 버전 (중간고사)**, 파이프라인 버전 (기말).
- 다루는 명령어: 메모리 `lw, sw` · 산술/논리 `add, sub, and, or` · 제어 `beq, j` (p.4엔 `slt`도 포함).

> 시험 포인트: 교수님 "중간에서는 이 명령어들만 써야 한다. 다른 명령어를 쓰면 0점." 수업에서는 "seven instructions"라고 말했는데, 슬라이드 목록은 lw, sw, add, sub, and, or, beq, j. 단순 데이터패스에서 실제로 다룬 건 R-type, lw, sw, beq.

> **수업 메모** `06주-1`
> 'before midterm, you will actually deal with the simplified version of the CPU, and for your final, you will deal with the pipeline version'. 중간엔 수업에서 정한 명령어만 쓰고, 다른 걸 쓰면 0점.

§ 명령어 실행 단계

1. **PC → instruction memory**, 명령어 fetch.
2. **Register numbers → register file**, 레지스터 읽기.
3. 명령어 종류에 따라:
   - **ALU**로 계산: 산술 결과 / load·store용 **메모리 주소** / **분기 대상 주소**
   - load·store면 **data memory** 접근
   - **PC ← 분기 대상 주소 또는 PC + 4**

명령어는 1 word = **32 bit = 4 byte** → 다음 줄로 가려면 **PC + 4**.

> 함정: 6주-1 녹음에서 "word = 32 bytes"라고 말한 부분이 있는데 말실수. 32 **bits** = 4 bytes.

§ CPU 개요와 MUX가 필요한 이유

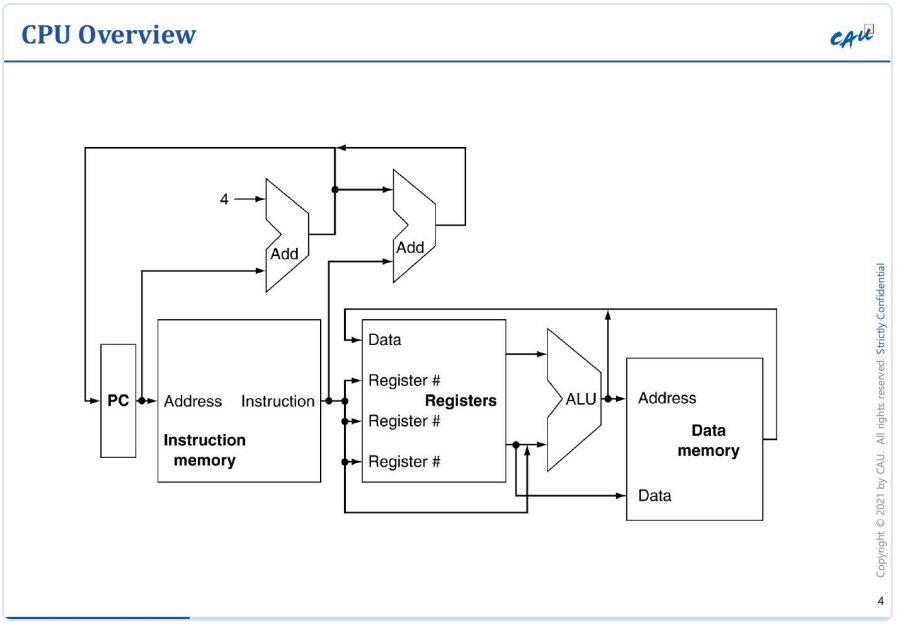
p.5 개요도: PC → Instruction memory → Registers → ALU → Data memory, 그리고 PC+4 adder와 branch adder.

p.6: **"Can't just join wires together: Use multiplexers"** — 서로 다른 출처의 선을 그냥 합칠 수 없으니 선택기가 필요한 곳 3군데:

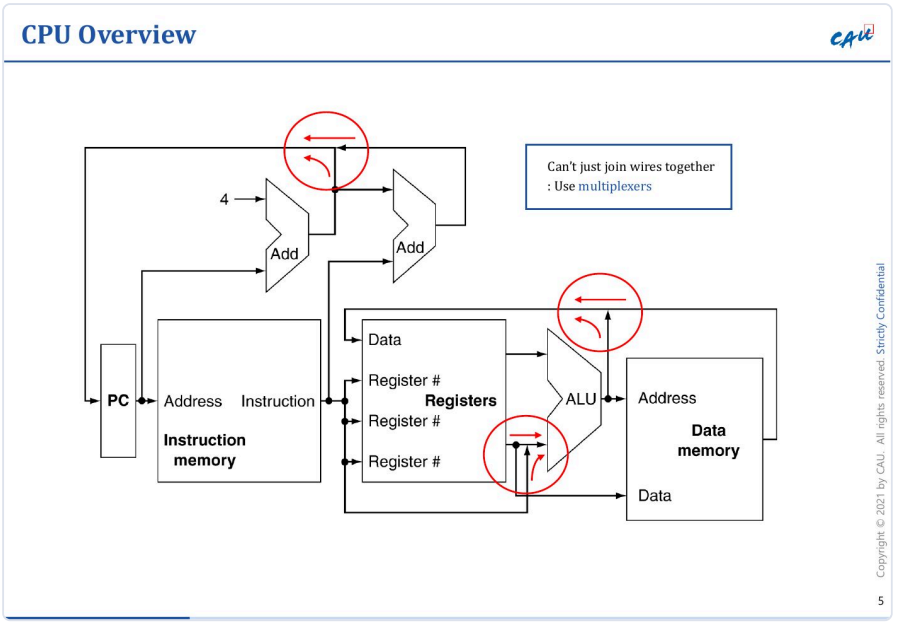
1. **PC 입력**: PC + 4 또는 branch target
2. **ALU 두 번째 입력**: 레지스터 값 또는 명령어의 16-bit 상수(sign-extended)
3. **레지스터 쓰기 데이터**: ALU 결과 또는 data memory에서 읽은 값

p.7: 여기에 **Control** 유닛이 붙어서 MUX 선택 신호, RegWrite, MemRead/MemWrite, ALU operation, Branch를 만든다.

> 교수님 설명: R-format이면 ALU 결과가 원하는 값이지만, lw면 ALU 결과는 **주소일 뿐**이고 실제 원하는 건 그 주소의 메모리 값. 그래서 쓰기 데이터 MUX가 필요.

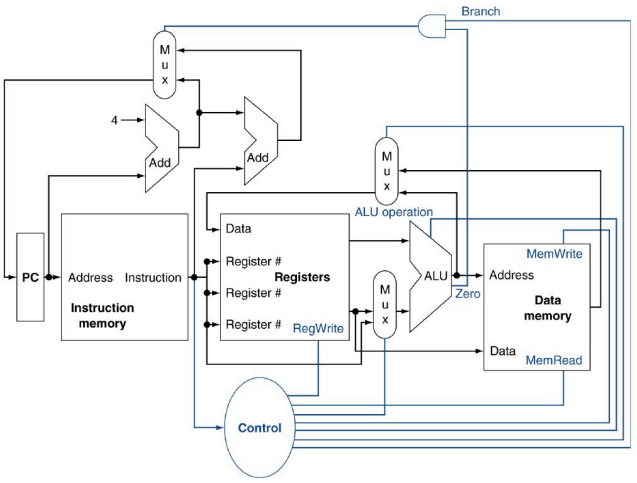


week6 p.5 · CPU Overview



week6 p.6 · Can't just join wires together: MUX가 필요한 3곳

CPU Overview: Control



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

6

week6 p.7 · CPU Overview: Control

논리 설계 기초 · 클럭

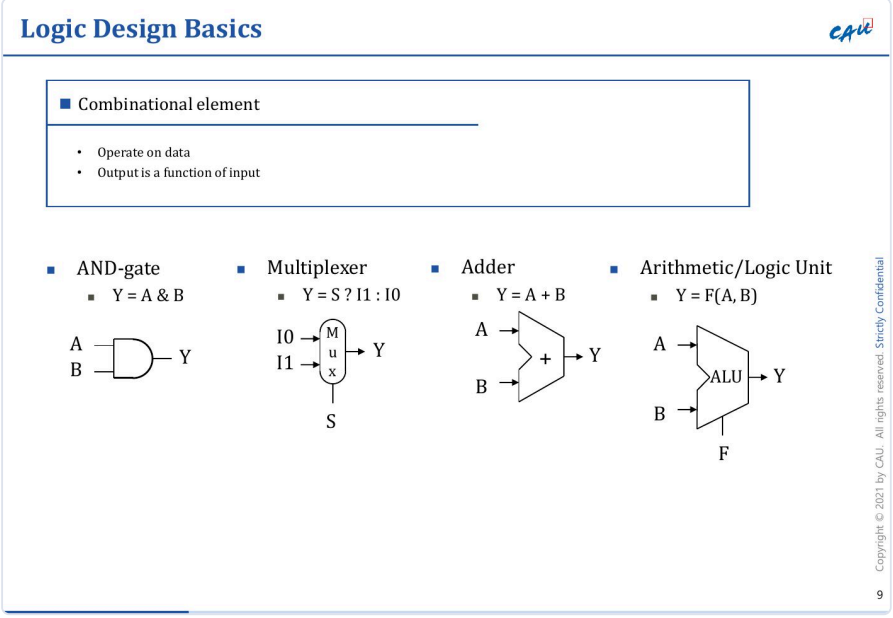
교안 범위: week6 p.8–15

§ 정보 표현과 조합 회로

- 이진 표현: low voltage = 0, high = 1. **선 하나 = 1 bit**, 여러 bit는 multi-wire bus (32-bit면 선 32개).
- **Combinational element**: 데이터에 연산. **출력이 입력만의 함수** (기억 없음).

소자	식
AND gate	Y = A & B
Multiplexer	**Y = S ? I1 : I0**
Adder	Y = A + B
ALU	Y = F(A, B), F = 수행할 함수

ALU는 adder의 확장판. R-format이면 F는 명령어의 맨 아래 6 bit(**funct**)로 결정됨.



week6 p.10 · AND, MUX, Adder, ALU

§ Multiplexer 읽는 법

- 입력선마다 번호가 붙어 있음 (I0, I1).
- **선택선 S = 0 → I0, S = 1 → I1**.
- 시험에서는 **어떤 선이 0이고 어떤 선이 1인지 그림에 표시돼서** 나옴. 교수님이 판서로 0과 1을 직접 써서 설명.

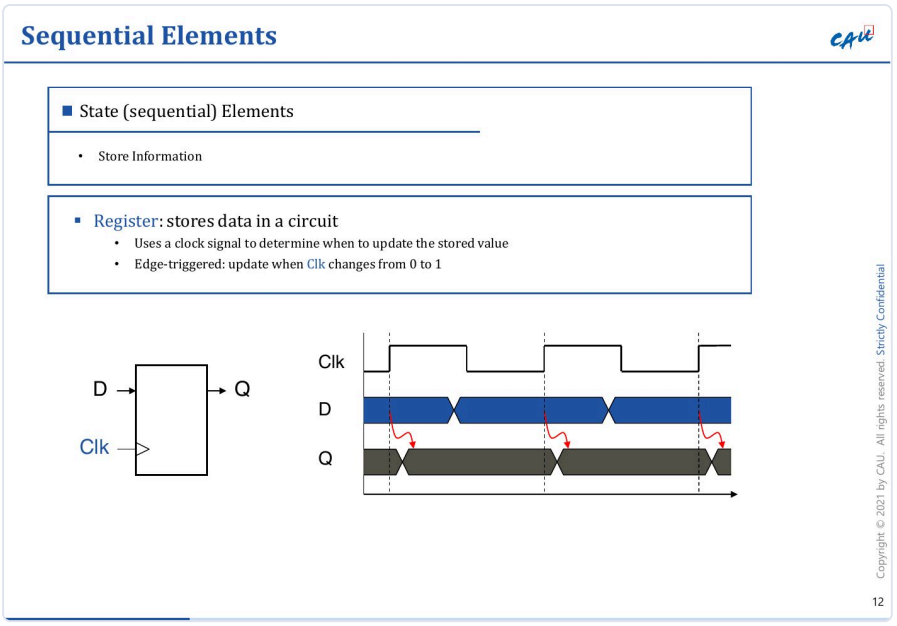
> 시험 포인트: 신호값은 "위/아래"가 아니라 **MUX에 적힌 숫자**로 정해. 숫자가 뒤집혀 있으면 신호도 반대 (6주-2 "be extremely careful on what is actually written on the multiplexer").

§ 순차 소자: 레지스터

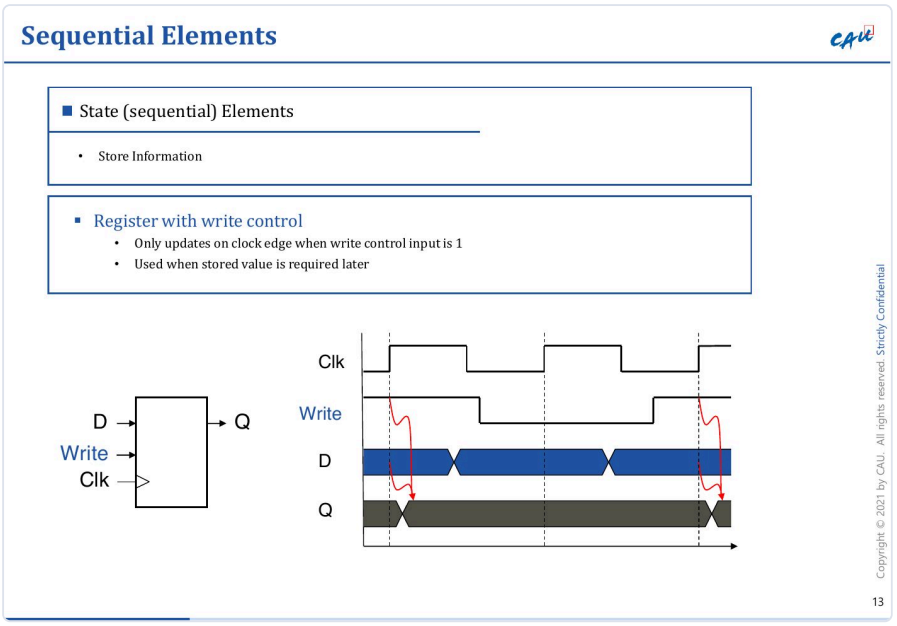
- **State (sequential) element**: 정보를 **저장**.
- **Register**: 회로에 데이터를 저장. **clock**으로 언제 값을 갱신할지 결정.

- **Edge-triggered**: Clk이 **0 → 1로 바뀔 때(rising edge)** 갱신.
- **Register with write control**: clock edge이면서 **write control 입력이 1일 때만** 갱신. 나중에 값이 필요할 때 사용. (레지스터 파일의 RegWrite가 이것)

함정: 6주-1에서 "0 to 1 or 1 to 0"이라고 말했지만 슬라이드와 6주-2 설명은 **0 → 1**. 시험은 슬라이드 기준.



week6 p.13 · Edge-triggered register



week6 p.14 · Register with write control

§ Clocking methodology

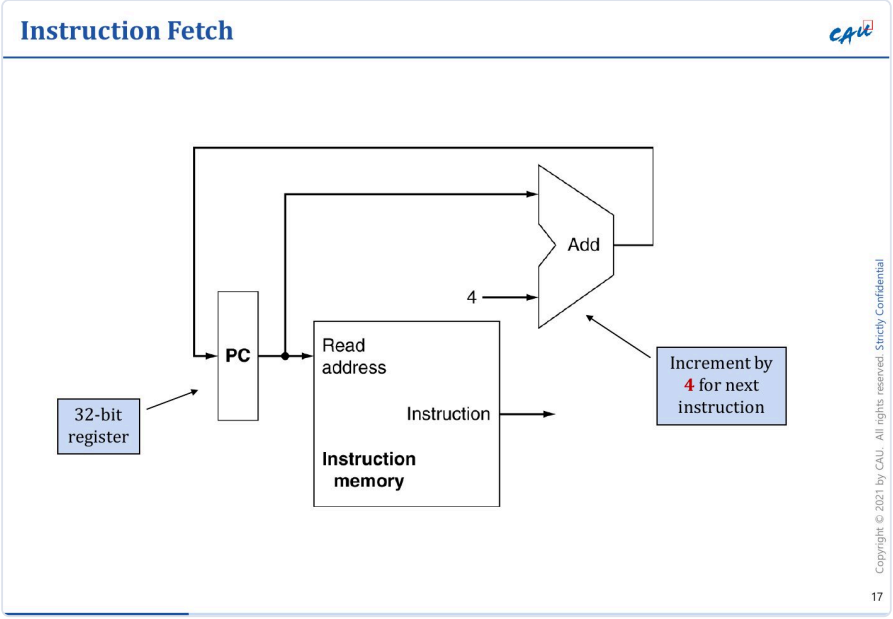
- 조합 회로는 **클럭 사이클 동안(클럭 edge 사이)** 데이터를 변환.
- 입력은 state element에서 오고, 출력은 state element로 감.
- **가장 긴 지연이 clock period를 결정**.

데이터패스 구성 요소

교안 범위: week6 p.16–24

§ 데이터패스 만들기 · Instruction fetch

- **Datapath**: CPU에서 데이터·주소를 처리하는 요소들 (레지스터, ALU, MUX, 메모리…). 명령어 부분집합에 맞춰 단계적으로 만든다.
- **Instruction fetch**: **PC(32-bit register)**가 instruction memory의 read address로 → 명령어 출력. 동시에 PC가 **Add**에 들어가서 **+4** → 다음 명령어 주소가 다시 PC로.



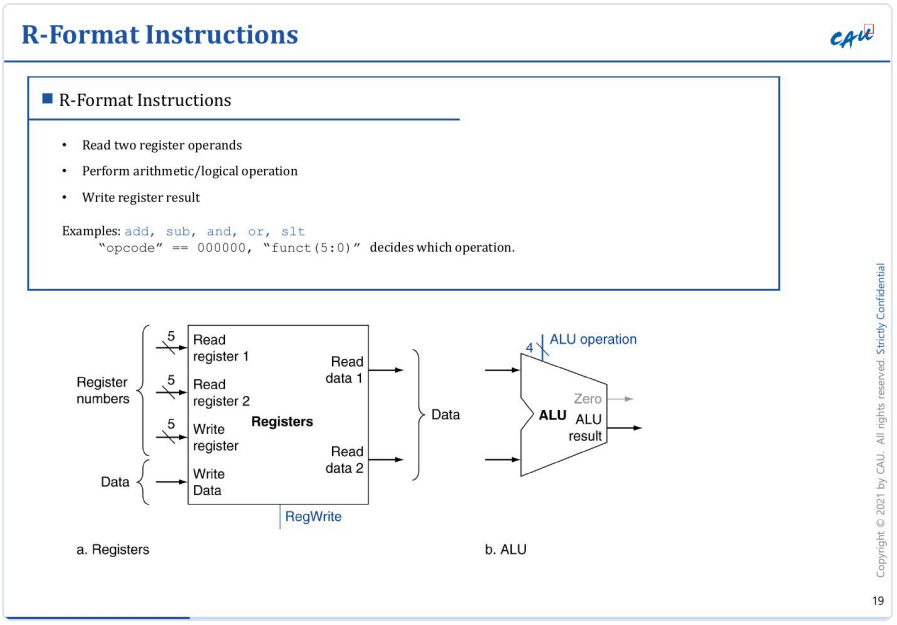
week6 p.18 · Instruction Fetch

§ R-format 데이터패스

R-format(add, sub, and, or, slt): opcode = 000000, **funct(5:0)**가 연산 결정.

1. 레지스터 피연산자 **2개 읽기** (Read register 1 = rs, Read register 2 = rt, 각 5 bit)
2. 산술/논리 연산 (ALU, ALU operation 4 bit)
3. 결과를 레지스터에 **쓰기** (Write register = rd, Write data, **RegWrite = 1**)

예 `add $t0, $s1, $s2` → op 0 | rs 17 | rt 18 | rd 8 | shamt 0 | funct 32.



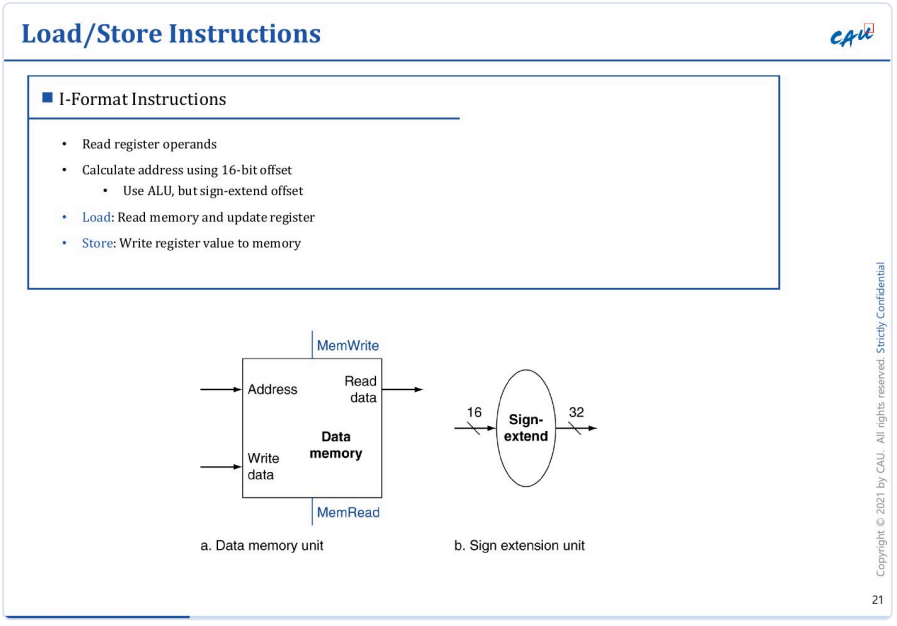
week6 p.20 · Registers와 ALU

§ Load/Store 데이터패스

1. 레지스터 읽기 (rs = base)
2. 16-bit offset으로 주소 계산: ALU 사용, offset은 **sign-extend (16 → 32)**
3. **Load**: 메모리 읽고 레지스터 갱신 (**MemRead = 1, RegWrite = 1**)
4. **Store**: 레지스터 값을 메모리에 쓰기 (**MemWrite = 1**)

Sign extension: 위 16 bit를 상수의 sign bit로 채움 (양수 0, 음수 1).

함정: 6주-1 23:18에 lw 설명하면서 "memory write field를 켠다"고 말했는데 말실수. lw는 **MemRead**와 RegWrite, MemWrite는 sw. 바로 뒤(24:50)에 정정함.



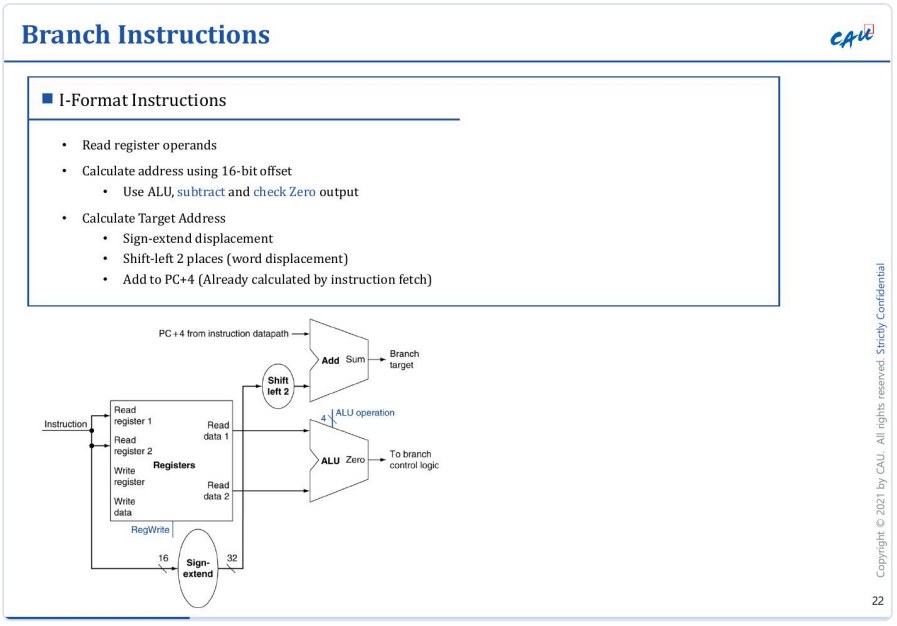
week6 p.22 · Data memory와 Sign-extend

§ Branch 데이터패스

beq:

1. 레지스터 피연산자 읽기 (rs, rt)
2. ALU로 **빼기** → **Zero 출력** 확인 (같으면 Zero = 1)
3. 분기 대상 주소 계산:
   - 변위(16 bit)를 **sign-extend**
   - **Shift left 2** (word 변위 → byte, ×4)
   - **PC + 4**에 더함 (PC + 4는 fetch에서 이미 계산됨)

> 시험 포인트: 분기 상수는 주소가 아니라 **(PC + 4)부터 몇 줄(명령어) 떨어졌나**. byte가 아니라 줄 수로 주는 이유는 4로 안 나눠지는 값이 들어가면 심각한 오류가 나기 때문.



week6 p.23 · Branch datapath

§ 요소 합치기: 4개의 MUX

- 단순 데이터패스는 **명령어 하나를 한 클럭 사이클에** 실행.
- 각 요소는 한 번에 한 기능만 → **instruction memory와 data memory를 분리**.
- 출처가 둘인 곳마다 MUX:
  - ALU 입력: 레지스터 vs 명령어의 immediate → **ALUSrc**
  - 레지스터 쓰기 데이터: ALU 출력 vs data memory → **MemtoReg**
  - 다음 PC: PC + 4 vs branch target → **Branch AND Zero** (PCSrc)
  - 쓰기 레지스터 번호: **Rt vs Rd** → **RegDst**

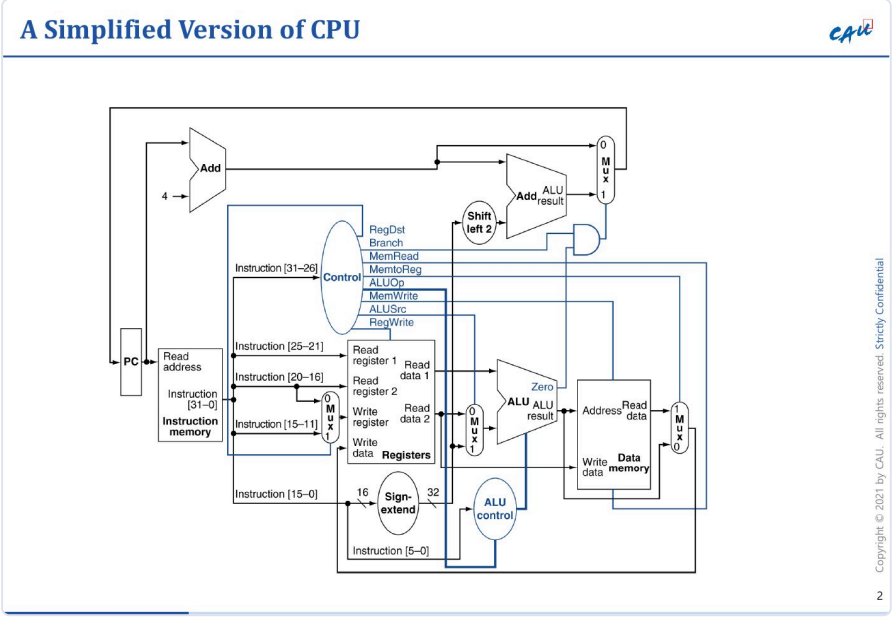
전체 데이터패스 따라가기 (R-type · lw · sw · beq)

교안 범위: week6 p.3, p.26, p.34–37

§ 명령어 필드가 가는 곳

p.3 그림에서 32-bit 명령어가 쪼개지는 곳:

필드	비트	가는 곳
opcode	[31–26]	**Control**
rs	[25–21]	Read register 1 (항상 읽음)
rt	[20–16]	Read register 2 **그리고** RegDst MUX의 0번 입력
rd	[15–11]	RegDst MUX의 1번 입력
imm	[15–0]	**Sign-extend** (16 → 32)
funct	[5–0]	**ALU control**



week6 p.3 · A Simplified Version of CPU (교수님이 '이 한 장을 완벽히' 이해하라고 한 슬라이드)

§ R-type (add $t0, $s1, $s2)

- opcode 000000 → Control이 R-type 신호를 냄.
- rs, rt 읽기 → ALU의 두 입력 모두 **레지스터** → **ALUSrc = 0**.
- 결과를 **rd(세 번째 레지스터)**에 씀 → RegDst MUX에서 [15–11]을 고름 → **RegDst = 1** (p.3에선 1번 입력이 rd).
- 쓰기 데이터는 **ALU 결과** → **MemtoReg = 0**. **RegWrite = 1**.
- 메모리 접근 없음 → MemRead = MemWrite = 0. Branch = 0. ALUOp = 10 (funct가 연산 결정).

§ lw ($t0, offset($s1))

- rs = base 읽기, 상수 offset을 sign-extend → ALU 두 번째 입력은 **상수** → **ALUSrc = 1**.
- ALU는 **덧셈**으로 주소 계산 (ALUOp = 00). ALU 결과는 값이 아니라 **주소**.

- Data memory 읽기 → **MemRead = 1**.
- 읽은 값을 레지스터에 → **MemtoReg = 1**, **RegWrite = 1**.
- 쓰는 레지스터는 **rt(두 번째)** → **RegDst = 0**.

---

§ sw ($t0, offset($s1))

- 주소 계산은 lw와 같음 → **ALUSrc = 1**, ALUOp = 00.
- rt의 값(Read data 2)이 data memory의 **Write data**로 → **MemWrite = 1**.
- 레지스터에 쓰지 않음 → **RegWrite = 0**, 그래서 RegDst와 MemtoReg는 **X (don't care)**.

---

§ beq (rs, rt, offset)

- opcode → Control이 **Branch = 1**.
- 두 레지스터를 **빼서** 비교 → ALU 입력 둘 다 레지스터 → **ALUSrc = 0**, ALUOp = 01 (빼기).
- 같으면 ALU **Zero = 1** → **AND(Branch, Zero) = 1** → PC MUX가 **branch target(1번)** 선택. 다르면 0 → **PC + 4(0번)**.
- 분기 대상: offset sign-extend → shift left 2 → (PC + 4)에 더함.
- 레지스터 쓰기 없음 → RegWrite = 0, RegDst = X, MemtoReg = X. 메모리 접근 없음.

---

§ MUX 숫자 함정 (p.3 MemtoReg)

p.3 그림의 MUX 3개는 **0이 위**인데, 맨 오른쪽 **MemtoReg MUX는 1이 위**(Read data), **0이 아래**(ALU result).

- 교수님 설명: "ALU 결과를 고르려면 0, data memory 값을 고르려면 1" → 그림 숫자와 일치. 위/아래가 아니라 **숫자**를 봐야 함.
- 교수님: "if I flip the numbers on your multiplexer … you will have to generate a total opposite signal". 시험 그림에서 숫자가 바뀌면 답도 바뀜.

> 시험 포인트: MUX 문제를 풀 땐 ① 원하는 입력선 찾기 ② 그 선에 적힌 숫자 = 신호값. 표를 외워서 쓰지 말 것.

---

§ 분기 오프셋 계산 (판서 예제)

Offset = (Target − (PC + 4)) / 4, **Target = (PC + 4) + Offset × 4**

6주-2 판서: PC = 108 → PC + 4 = 112, 목표 = 124

```
(124 - 112) / 4 = 3          → beq rs, rt, 3
3 (16-bit) → sign-extend → 0000 … 0011 (32-bit)
shift left 2 → 12
112 + 12 = 124  ✓
```

6주-1 판서: 100번지의 beq, 목표 180 → (180 − 104) / 4 = 76 / 4 = **19**.

- 목표가 위쪽(이전 코드)이면 **음수 오프셋** → sign extension으로 위 비트가 1로 채워짐.

ALU control

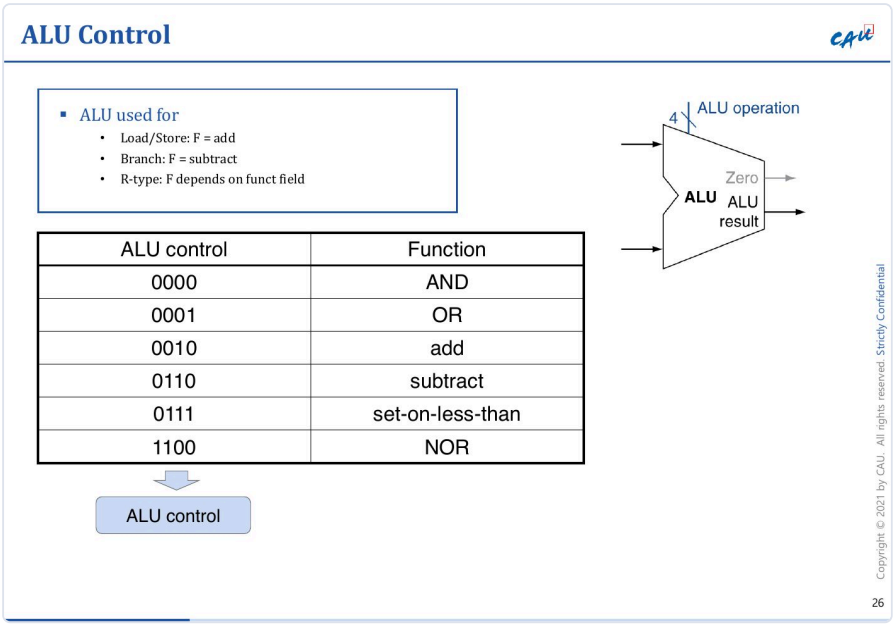
교안 범위: week6 p.27–30

§ ALU의 쓰임과 ALU control 코드

- Load/Store: F = **add** (주소 계산)
- Branch: F = **subtract** (같은지 비교)
- R-type: F는 **funct 필드**에 따라

ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set-on-less-than
1100	NOR

> 시험 포인트: 교수님 "이 표는 외울 필요 없다. 시험에 주어진다. 대신 **읽고 회로에 적용**할 줄 알아야 한다." (6주-2 37:31, 어느 표를 가리켰는지는 녹음상 추정)



week6 p.27 · ALU control 코드

§ ALUOp와 funct로 ALU control 만들기

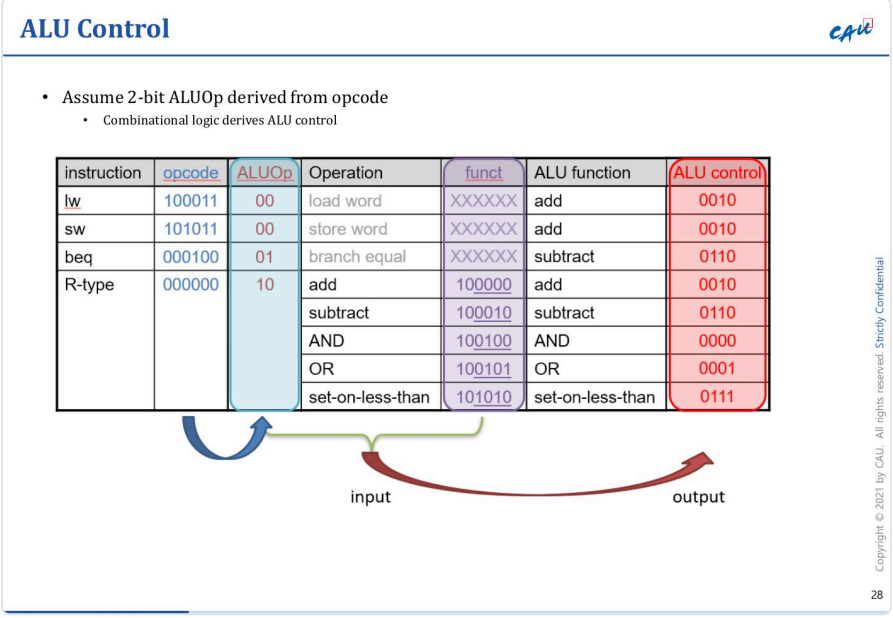
2-bit **ALUOp**는 opcode에서 (Main control이) 만들고, ALU control 유닛이 **ALUOp + funct(6 bit)**로 4-bit ALU control을 만든다 (조합 논리).

opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	0010

opcode	ALUOp	Operation	funct	ALU function	ALU control
sw	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	set-on-less-than	0111

- lw/sw/beq는 I-type이라 **funct가 없음** → ALUOp만 보고 결정 (00 → 더하기, 01 → 빼기).
- ALUOp = 10이면 ALU control이 **funct를 기다렸다가** 결정 (add = 32 = 100000).

함정: R-type funct들은 전부 `10`으로 시작(100000, 100010, ...)해서 R-type ALUOp `10`과 앞 두 비트가 같다. 교수님이 "헷갈리지 말라"고 짚은 부분 (6주-2 42:08).



week6 p.29 · ALUOp + funct → ALU control

§ K-map으로 구한 ALU control 논리

ALU control 표를 진리표로 놓고 K-map으로 간단히 하면:

```
Op3 = 0
Op2 = ALUOp0 + ALUOp1·F1
Op1 = ¬ALUOp1 + ¬F2
Op0 = ALUOp1·(F3 + F0)
```

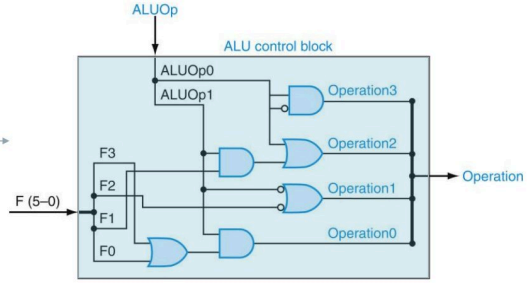
함정: 교안 p.30에는 NOT 표시가 빠져서 `Op1 = ALUOp1 + F2`로 적혀 있음 (Op3 줄의 `APUOp0`도 오타). 그대로 쓰면 lw(ALUOp 00)에서 Op1 = 0이 나와 표(0010)와 안 맞는다. 위 식은 표의 8행 전부와 일치하는지 검산한 것.

ALU Control

ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
0	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

Karnough-Map!

Op3 = always 0
= ALUOp0 · APUOp0
Op2 = ALUOp0
+ ALUOp1 · F1
Op1 = ALUOp1 + F2
Op0 = ALUOp1 · (F3 + F0)



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

29

week6 p.30 · K-map (식의 NOT 표시 누락에 주의)

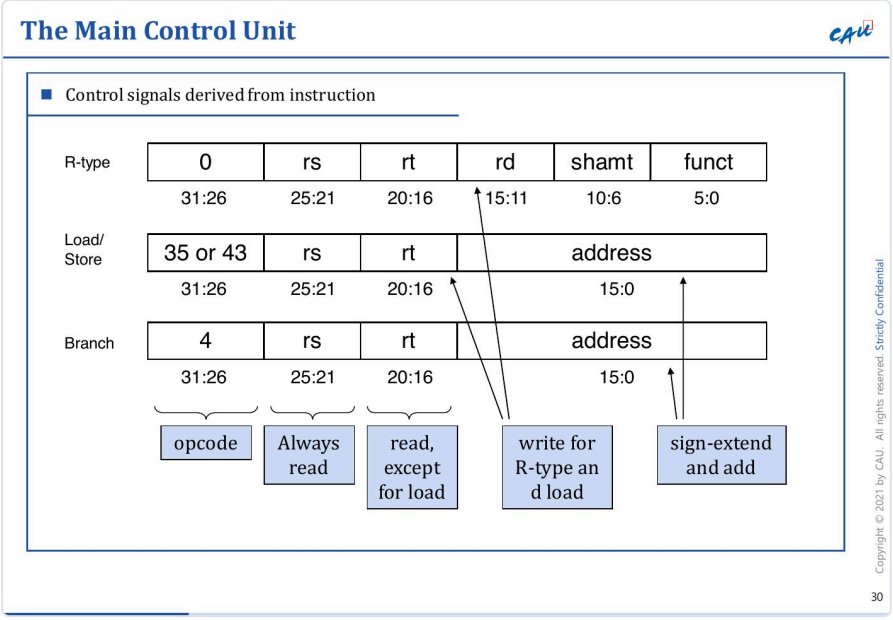
Main control unit · 제어 신호 표

교안 범위: week6 p.31–33

§ 명령어 형식과 제어의 관계

	[31:26]	[25:21]	[20:16]	[15:11]	[10:6]	[5:0]
R-type	0	rs	rt	rd	shamt	funct
Load/Store	35 or 43	rs	rt	address (15:0)		
Branch	4	rs	rt	address (15:0)		

- **opcode** → Control 입력
- **rs: 항상 읽음** (Always read)
- **rt: 읽음, 단 load에서는 쓰기 대상** (read, except for load)
- **rd: R-type에서 쓰기 대상** (write for R-type)... load는 rt에 씀 → RegDst MUX
- **address: sign-extend 후 더함**



week6 p.31 · The Main Control Unit

§ 제어 신호 표

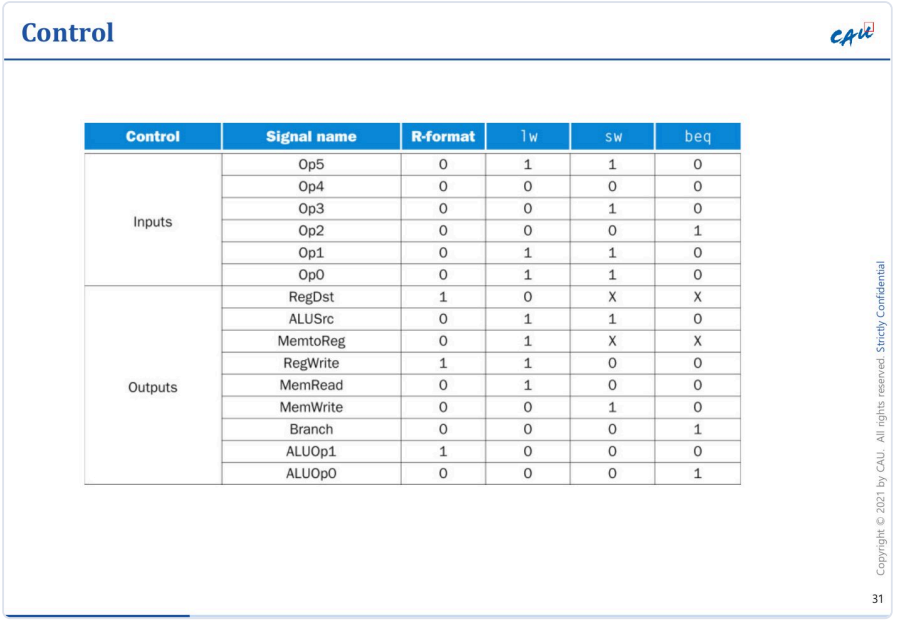
신호	R-format	lw	sw	beq
RegDst	1	0	X	X
ALUSrc	0	1	1	0
MemtoReg	0	1	X	X
RegWrite	1	1	0	0
MemRead	0	1	0	0
MemWrite	0	0	1	0

신호	R-format	lw	sw	beq
Branch	0	0	0	1
ALUOp1	1	0	0	0
ALUOp0	0	0	0	1

입력(opcode Op5–Op0): R-format 000000, lw 100011 (35), sw 101011 (43), beq 000100 (4).

외우지 말고 **이유로** 맞히기:

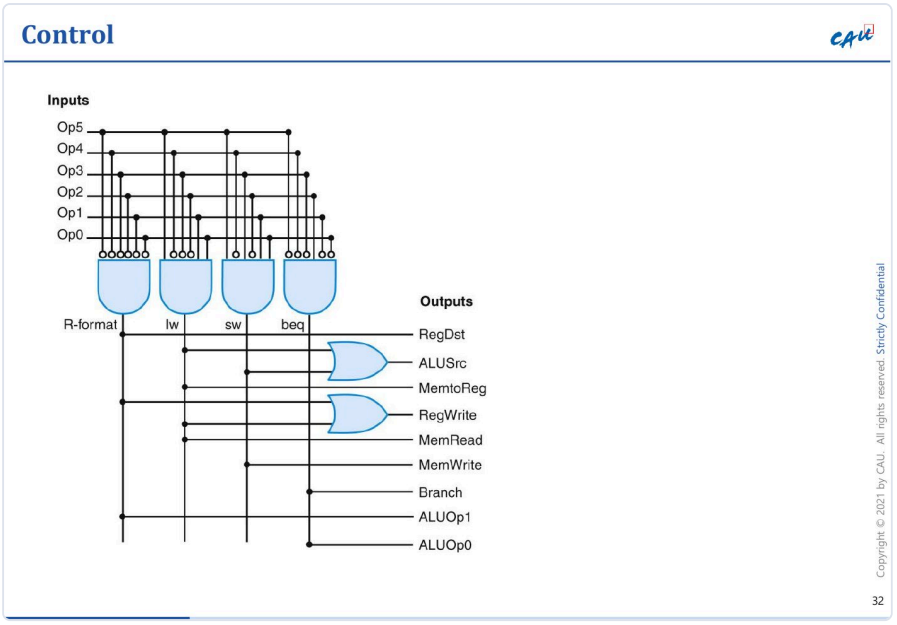
- **X가 나오는 곳 = 레지스터에 안 쓰는 명령어(sw, beq)의 RegDst·MemtoReg**. 쓰지 않으니 어떤 레지스터·어떤 데이터든 상관없음.
- ALUSrc = 1 ⇔ 상수(offset)를 쓰는 lw, sw.
- RegWrite = 1 ⇔ 결과를 레지스터에 쓰는 R-type, lw.



week6 p.32 · 제어 신호 표

§ 제어 유닛 회로 (p.33)

Opcode 비트 6개(Op5–Op0)를 AND 게이트 4개가 각각 **R-format / lw / sw / beq**로 디코딩하고, OR 게이트로 출력 신호를 만든다. 예: ALUSrc = lw OR sw, RegWrite = R-format OR lw. RegDst = R-format, MemtoReg = lw, MemRead = lw, MemWrite = sw, Branch = beq, ALUOp1 = R-format, ALUOp0 = beq.



week6 p.33 · 제어 유닛 회로

§ lw/sw offset은 상수 (교수님 반복 강조)

- lw/sw는 **I-type**이라 offset은 반드시 **상수**.
- 시험에서 offset을 레지스터 값으로 준 문장 → **False**.
- 레지스터에 든 offset을 쓰려면: 먼저 base + offset을 계산해서 레지스터에 넣고, `lw $t0, 0($t1)` 처럼 **상수 0**을 offset으로.

> 시험 포인트: 3주·4주·6주에 걸쳐 같은 경고. "you can never just provide your offset which is contained inside the register directly into an I-type instruction".